Learning to Navigate an N-Body Field

Defining, Measuring, and Learning Difficulty in a Game-Scale Gravitational Navigation Environment

Kevin Horecka*

November 1, 2026

Abstract

Navigating a gravitational field produced by several moving bodies is a problem with a long history in astrodynamics and a short one in reinforcement learning, and the two literatures describe what makes an instance of it difficult in different terms. Here, we examine a game-scale version of the task, in which a craft with a steerable engine and a finite Δv budget must reach a short sequence of moving targets inside a field generated by up to 32 bodies, and ask what it means for a level of this task to be hard, how that can be checked, and what a learning agent can be expected to do about it. Two ways of picturing the task are developed (a sequence of regions of differing predictability, inside the chaotic ones of which the aim is to bound the exit trajectory, and a space of models of the field from one-body conics to a policy learned with little structure imposed), and difficulty is treated as having a representational component, i.e. the degree to which a coarse one-body model of the field is wrong, and a reaction-time component (the width of the window in which an action must be taken), of which only the first is treated here. The representational component is given a closed form, $\mathcal{D} = \eta/A$, where η is the fraction of the local gravitational acceleration that a one-body model discards and A is the fuel budget expressed in units of the impulse gravity will deliver over the remaining flight, such that $\mathcal{D}$ is the share of the budget a wrong model would cost; it requires no simulation and inverts, so that a level can be constructed at a chosen difficulty. The field is shown to be quasi-integrable on the timescale of an episode (a Lyapunov time of roughly 140 s against a 150 s episode), so chaos has to be built into a level deliberately, and the episode clock is argued to be the least informative of the constraints, with the caveat that what a longer clock buys depends on the navigator: a trajectory optimizer converts timeouts into arrivals as the clock lengthens, while a scripted reflex pilot converts them into escapes and collisions, and a learned policy trained at the longer clock mostly fails to arrive at all unless its discount is raised to match, at which point it recovers the shorter clock's performance, so that the clock and the discount behave as a single setting. Finally, a proximal policy optimization agent trained across 20 runs (5 seeds per condition) is used both to evaluate the levels and to be evaluated by them: of four physically meaningful readouts added to its observation, the two that are algebraic combinations of quantities it already observes cost 3.1 to 10.1 percentage points of success depending on the evaluation tier (Welch's t on seed means, $p \leq 0.029$ on every tier), while the two that carry new information cost nothing, and the cost vanishes when the network's embedding width is doubled, so that it was a cost in capacity rather than in conditioning; the closed-form $\mathcal{D}$ predicts per-instance success ($\rho = -0.585$ over 2,388 instances) but does so through its fuel-budget denominator, the representational numerator adding nothing once the budget is known; and training for 10,000 updates instead of 3,000 raises success by 4.7

*Correspondence: kevin.horecka@gmail.com. Code, levels and every measurement reported here: https://github.com/kevroy314/spaceship-navigator.

to 15.3 points per tier, to 95.1%, 57.2%, 18.9% and 40.9%, which is larger than most of the effects the comparisons measure. Taken together, these results suggest that the difficulty of an n-body navigation task has several separable axes and that each has to be measured, and the environment, levels and trained policies are provided so that the measurement can be repeated and extended.

1 Navigating an n-body field

The motion of a small body in the gravitational field of several larger ones is among the oldest problems in dynamics and among the least tractable, with closed-form solutions available for two bodies and, under restriction, for three, and with the general case known since Poincaré to admit no such solution at all [1–3]. Spacecraft guidance has nonetheless managed the problem for some sixty years without addressing the entire possibility space of trajectories, since a trajectory is usually designed as a sequence of two-body arcs, each dominated by the nearest massive body and stitched to the next at the surface where dominance passes between them (the patched-conic approximation, which treats the field as locally Keplerian and the transitions as instantaneous [4–6]), and since gravitational encounters are used deliberately to change speed and direction at little or no cost in fuel [7]. Figure 1 draws three designs of this kind. Voyager 2 reached Neptune in twelve years on a flight that would have taken about thirty without the assists, each of its first three planetary flybys adding heliocentric speed [8, 9]; Parker Solar Probe lowered its perihelion over seven Venus encounters, each flyby removing speed instead [10, 11]; and Artemis I reached and left a distant retrograde orbit of the Moon by way of two powered lunar flybys [12]. In each case the design was judged chiefly on fuel, exposure and arrival, with elapsed time a constraint to be met rather than the quantity to be minimized, and in each case a large share of the change in velocity was supplied by the bodies themselves.

Reinforcement learning has a much shorter history with this sort of gravitational navigation problem. Learning-based guidance has been shown to recover near-optimal control on particular low-thrust transfers, in particular in the three-body setting [18–22], but, much as the historical designs above do, these reduced cases leave the wider possibility space of gravitational navigation unaddressed. Beginning to explore that space is difficult in part because it is unclear how the difficulty of a specific configuration of bodies should be defined, and without such a definition it is difficult to say how effective a given algorithm is at the broader task, as opposed to the instances it was shown, with overfitting to a particular n-body configuration the risk that follows [23, 24]. Generalization in this task therefore seems to require, before anything else, a better understanding of the task space itself, which is one of the primary goals of this paper and one it accomplishes only in part.

The task studied here is a particular subspace of that problem. A task instance is a simulation of N gravitating bodies for a time T, in which a craft with a steerable engine and a finite Δv budget must fly through, or match velocity with, a short sequence of targets, several of which may be moving bodies themselves, choosing an action at a control rate H, so that an instance admits HT decisions. The action space is discrete, which limits the dimensionality of the control problem (Appendix B notes the alternatives) and, which was the reason for the choice, confines the simulation to actions a person could perform on a gamepad or a similar interface, so that comparison with human performance remains possible. Figure 2 lays out one instance in full. The environment is built as a testbed for reinforcement learning, though the difficulties reported here are not specific to learning, since a search-based trajectory optimizer meets most of them as well (Section 3.5).

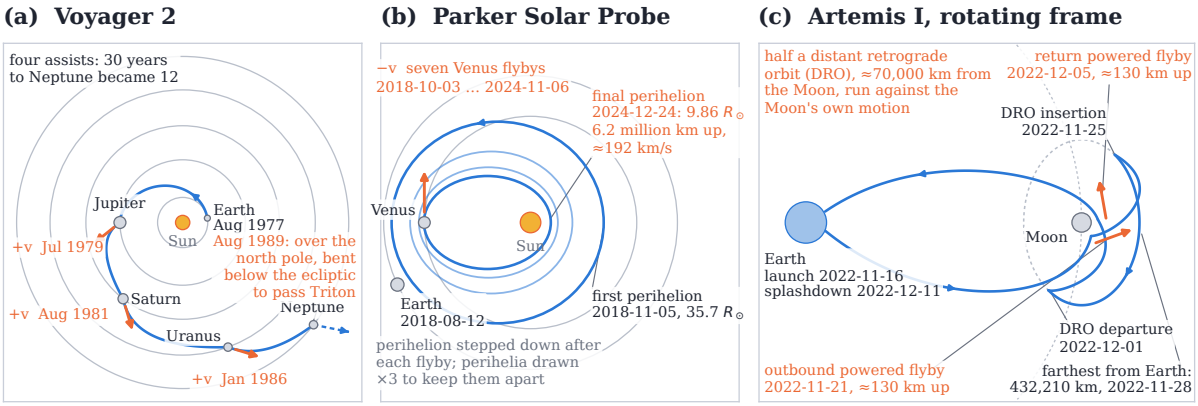


Figure 1: Three missions that solved an n-body problem by using the field. Schematic and not to scale; blue is the craft and orange marks the trick. (a) Voyager 2 (launched 1977-08-20) took gravity assists at Jupiter (July 1979), Saturn (August 1981), Uranus (January 1986) and Neptune (August 1989), the first three each adding heliocentric speed, which cut the flight to Neptune from about thirty years to twelve [8, 9, 13]; the Neptune encounter was spent on bending the path below the ecliptic to pass Triton. (b) Parker Solar Probe (launched 2018-08-12) used seven Venus gravity assists between 2018-10-03 and 2024-11-06, each removing heliocentric speed, to step its perihelion from $35.7\,R_\odot$ (2018-11-05) to $9.86\,R_\odot$ on 2024-12-24 [10, 11, 14, 15]; perihelia are drawn three times too large so that the inner orbits can be told apart. (c) Artemis I (launched 2022-11-16) in the Earth–Moon rotating frame: an outbound powered flyby about 130 km above the far side of the Moon (2022-11-21), a coast to a distant retrograde orbit about 70,000 km from the Moon (insertion 2022-11-25), half a revolution of that orbit, run against the Moon's own motion about Earth, departure (2022-12-01), a return powered flyby (2022-12-05) and splashdown (2022-12-11) [12, 16, 17]. Drawn by `scripts/fig_missions.py`.

The rest of this paper returns repeatedly to two ways of picturing the task. Each has its own utility in describing the task and each suggests a different notion of difficulty; neither appears sufficient alone, and part of what this paper does is to explore both in search of a perspective that is at once understandable and sufficient for training.

The first treats the craft's flight as passage through a sequence of regions of differing predictability, i.e. as a state machine whose states are regimes rather than positions. In a well-behaved regime a trajectory can be planned ahead with confidence, while inside a chaotic one the exit trajectory cannot be predicted beyond a few Lyapunov times, which is what it means for the region to be chaotic [25, 26], but it can be bounded, i.e. the region can be entered in a way that constrains the set of exits to something a plan on the far side can absorb, which is the construction that reachable-set and tube-based formulations of control make rigorous [27, 28] and that belief-state planning gives a probabilistic form [29]. On this view a chaotic region behaves as a noise filter, in that it maps the set of states a navigator can enter it with onto a set of exits whose spread the navigator can bound but whose particular member it cannot choose, so that the plan is made for what comes after the region and the region itself is treated as a source of bounded uncertainty. This picture is close to the way a close encounter is treated in trajectory design, where the quantity constrained is the state after the encounter, and it is the one that makes difficulty depend on what lies after a region as much as on the region itself.

The second picture is a space of models of the field, spanning from strategies a person would write down to a policy learned from experience with little structure imposed on it in advance. The analogy is to forecasting, where a mean model carries only distributional assumptions, an

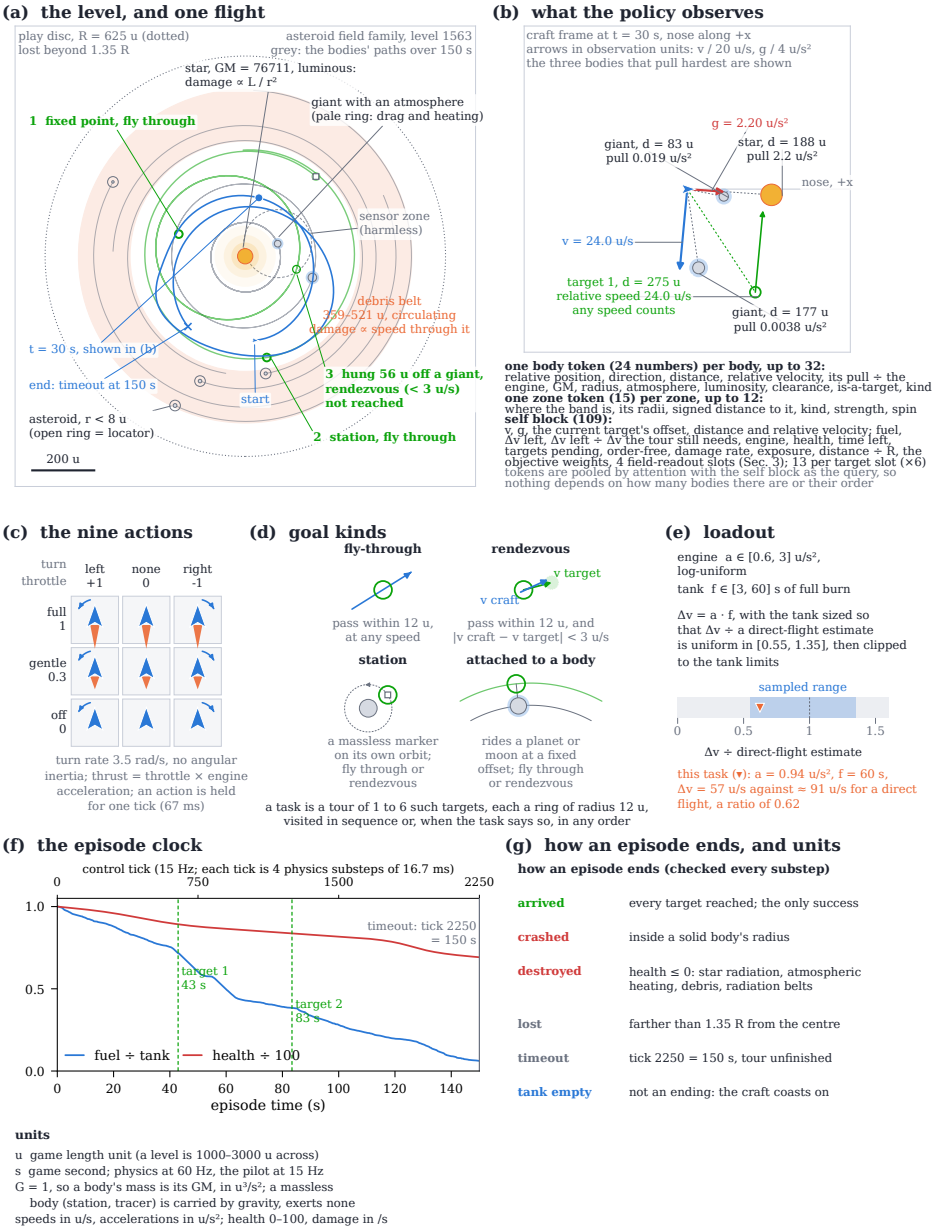


Figure 2: The task. (a) One level of the `asteroid_field` family (validation level 1563, radius 625 u) with the bodies' paths over one 150 s episode: a luminous star, two giants with atmospheres, a circulating debris belt, asteroids drawn with locator rings since their true radius is below 8 u, and a three-target tour (a fixed flythrough, a station flythrough, and a rendezvous hung 56 u off the outer giant). The flight shown is one of 32 sampled rollouts of a trained policy, all of which reach the first two targets (at 43 s and 83 s) and then circle the star to the clock without matching the giant's velocity, and it is shown here as a failure. (b) What the policy observes at $t = 30$ s, in the craft's frame: its velocity and the local gravity, the three bodies that pull hardest, the current target, and the contents of the body, zone and self tokens (Appendix A). (c) The nine discrete actions, turn × throttle. (d) The four goal kinds and their tolerances. (e) The loadout, with the Δv budget as a multiple of the direct-flight estimate and this task's value marked. (f) Fuel and health of the flight in (a) against the 2250-tick clock, with the two arrivals marked. (g) The five ways an episode ends, the one condition (an empty tank) that does not end it, and the units. Drawn by `scripts/fig_task.py`.

autoregressive model adds structured priors about how a series depends on its own past, and a transformer carries assumptions of a different kind, about which parts of a series are allowed to inform which, and comparatively few about the form of the dependence [30, 31]; the models differ in which priors they commit to, and a practitioner samples the space at points suggested by prior methods and at points more general than any of them. At the structured end of the space for this task sit dead heading (point at the target and burn), adequate when gravity is negligible against the engine; leading a moving target; conic reasoning, in which only the nearest massive body is accounted for and a flight is a sequence of Keplerian arcs [4]; and the deliberate use of encounters for assists [7] and of the field's own transport structure for low-energy transfers [32, 33]. Between these and the unstructured end lie methods that learn a model of the dynamics and plan within it [34], and at the unstructured end sits the plain reinforcement-learning loop, in which the craft observes the field, acts, is rewarded for progress, and improves a policy by gradient steps on that reward [35, 36], committing to no representation in advance. The space is not a line, since priors can be added along more than one axis, but each point in it is a representation of the field, and a level is hard on this picture when the representation a navigator carries is the wrong one for it. The unstructured end has a difficulty of its own, which arises when the craft enters a region in which nearby trajectories diverge exponentially, since the objective a policy gradient climbs then becomes rough at every scale below the divergence length and carries no usable descent direction unless the discount and control interval sit on the right side of a threshold set by the field's Lyapunov exponent [37–39]; whether this environment's field crosses that threshold is measured in Section 3.4. Of the two pictures, this is the one that yields a closed-form difficulty measure, developed in Section 3.2.

The reason to want such a measure is the size of the space. The set of possible configurations of gravitating bodies is, for practical purposes, unbounded, and configurations differ in kind as well as in degree, since each produces a phase-space portrait whose properties (where one body dominates, where trajectories diverge, where a hazard lies) bear on how hard a learner or a person finds the task. A formal scale on which configurations can be ranked by difficulty would allow learning to be bootstrapped, for a person or an agent, through a pedagogical progression from levels that a simple representation handles to levels that defeat it [40, 41], and the shape of that progression is itself informative about the task. This paper is the beginning of a wider exploration of control, under particular task objectives, in a setting in which the field itself moves on the timescale of a flight, and it is written as an invitation to attempt the task alongside the attempt reported here.

2 The problem of defining "difficulty"

A level generator that is asked for a hard level has to decide what "hard" means, and the two pictures above suggest that no single answer exists [42–44]. The reasons a level can be hard are better derived from the structure of the task than collected by example. A task instance asks a craft to carry its state (position, velocity, time, remaining fuel, remaining health) from a start into a sequence of target sets, in order, and whether it can is a question of reachability under the dynamics and the available control [27, 45]. The structure underneath is a graph whose nodes are regions of the craft's state space, whose edges are the transitions the field and the engine admit between them, each carrying a cost in fuel, time and exposure, and in which the task is a path from the start through each target set in turn whose accumulated costs stay within their budgets. A free-order tour adds a combinatorial choice of the order in which the target sets are visited.

The ways a level can be hard are then, as far as this analysis reaches, the ways that path can

fail to exist, fail to be found, or fail to be flown, and the three are distinct. A path can fail to exist because a budget is exceeded, since the fuel, the clock and the craft's tolerance for damage are each a cap on the path's accumulated cost, or because the geometry of the targets makes every path expensive, since targets can be placed far apart, spread so that every ordering is long, or set in regions the dynamics make costly to reach, so that the required cost exceeds any budget the loadout allows. A path can exist and fail to be found because the navigator's representation of the edge set differs from the true one, which is what happens when a coarse model of the field is wrong (the second picture), or because the edges out of a region are distributions over exits, which is what an encounter that scatters nearby trajectories does to the graph (the first picture), so that nothing on the far side can be planned for. A path can also be found and fail to be flown because the window in which a decisive action must be taken is narrower than the controller can hold. A level that is hard for one of these reasons may be easy for every other, and the repair for one does nothing for another, since widening a timing window does not change a contested field and simplifying the field does not widen a window; the quantities of Section 3 are each aimed chiefly at one of these reasons and are never combined into a single number for that reason. Interestingness, which is usually what a level designer wants, is related to difficulty without being identical to it, since a level that is impossible is not interesting and a level that is hard for an uninteresting reason (the clock, as Section 3.6 argues) is merely frustrating. None of these notions, moreover, can be checked by exhaustion.

Consider the most naive possible procedure for establishing that a level is solvable, which is to enumerate every control sequence and simulate each one. The control at each of T steps is a direction and a thrust magnitude, both continuous, so even with a discretization to a distinct actions per step the number of sequences is a^T, and at $T = 2250$ this is beyond reach for any $a > 1$; the discrete nine-action set used here does not rescue the situation (9^{2250} has more than two thousand digits), and the underlying continuous problem has no finite enumeration at all. Formal results in motion planning make the same point: the generalized mover's problem is PSPACE-hard [46–48], and remains so with moving obstacles even for a point robot of bounded velocity [49]; finite-horizon planning under partial observation is PSPACE-complete and its infinite-horizon form undecidable [50, 51]; and reachability for quite simple classes of continuous and hybrid dynamics is undecidable [52–54]. Note that these are results about problem classes rather than about any single instance, and that approximate search is not hopeless in principle, since kinodynamic planning with bounded velocity and acceleration admits a polynomial-time ε-approximation [55]; what they establish is that no efficient general procedure is to be expected, and that nothing about the field studied here, which couples every body's position to the craft's acceleration at every step, places it among the restricted cases for which an approximation is known.

Exhaustive checking is therefore replaced, in practice, by one of two things, a simplifying assumption (a coarse model of the field, a restricted family of control sequences, a short horizon) or a search that samples the space and reports what it finds, and both are used in this paper. Both also share a limitation that a chaotic regime makes acute. In a region where nearby trajectories diverge exponentially, the numerical trajectory of a given control sequence and the true trajectory of that sequence part company within a few multiples of the Lyapunov time, since the shadowing of a computed orbit by a true one breaks down in finite time when the local expansion rate fluctuates [56–58], so that a check which reports that a sequence arrives is a statement about a trajectory that may correspond to nothing the craft can fly, and the precision of the check becomes the limit before its cost does. The same literature supplies the premise on which this paper's method rests, which is that ensemble statistics over many orbits remain reliable where

individual orbits do not [59, 60]. Thus, in general, a level's difficulty cannot be read off from its definition and has to be measured over a population, and the measurement is itself only as good as the probe used to make it. The rest of the paper describes the simplifications made here to render that measurement tractable, and what they show.

3 Heuristics for difficulty and interestingness

The quantities used in this paper to describe a level are heuristics, in the sense that each is a cheap proxy for a property that cannot be computed exactly, and the reader is asked to hold them as such. What follows is the environment in enough detail to define them, then the quantities, then the properties of the field that bear on whether they mean anything. Exposure, which Section 2 names as a cap on the path and Section 3.6 ranks as the most interesting of the constraints, has no quantity of its own here and is measured only through the endings it produces.

3.1 The environment

The experiments reported here use $N \leq 32$ bodies, $T = 150\,\mathrm{s}$ and $H = 15\,\mathrm{Hz}$, so that an instance admits 2250 decisions, each one of nine actions (three turn directions crossed with three throttle levels). Table 1 lists the modelling assumptions and fixed parameters of the simulation, and the ones that shape the results most are set out here. The simulation is planar with gravitational constant $G = 1$, in game units (u) and game seconds (s), with a typical level spanning 1000–3000 u. Bodies attract one another and are advanced by a symplectic leapfrog integrator of the family standard in n-body work [61, 62], and the craft is a test particle (it is accelerated by every body and accelerates none), so that a level's body trajectories are fixed by its initial snapshot, which is what allows a level to be reproduced from a seed, replayed exactly, and integrated independently by a browser client from a snapshot of about 1.5 KB. Four physics substeps make one control tick, so the craft and a human player both act at 15 Hz.

A task instance is a short tour of between one and six targets, each either a point in free space, a station, or a point carried by a moving body at a constant offset, with an arrival radius and a velocity tolerance (any speed for a flythrough target; a small relative speed for a rendezvous, which is materially harder and is drawn only for a target that moves), in a fixed or a free order. The craft's Δv budget and engine acceleration are drawn per instance from ranges chosen so that a craft can rarely ignore gravity and fly straight at its target. Hazards are radiation from luminous bodies, atmospheric heating, debris fields and radiation belts. An episode ends on arrival at the last target, collision, destruction by accumulated damage, departure past 1.35 times the level radius, or the clock; an empty tank is not an ending, since the craft coasts on and the episode runs to the clock, which is one of the reasons the clock matters as much as it does (Section 3.6). Appendices A and C give the observation vector and the goal conditions in full.

Levels are generated procedurally in twelve families, listed in Table 2 with the reason each is in the set. The families are samples from the space of configurations, chosen to span the regimes a navigator would meet, and no taxonomy of the field stands behind them; they are the means by which an understanding of the parameters that define a level is bootstrapped here, since the quantities of the following subsections were developed by measuring them on these samples, and the families are in turn placed on those quantities (Table 2, Figure 3), so that a parameter earns its place by separating families that were chosen on other grounds. The first such parameter is the mass hierarchy, i.e. whether one body dominates the field nearly everywhere, somewhere, or nowhere: three families (`sol_like`, `jovian`, `saturnian`) are hierarchical systems in which one

Assumption	Value or rule	Consequence
Dynamics	planar; $G = 1$, so a mass is its parameter GM;	the field is a 2-vector at every point; bodies move visibly within one episode
geometry	lengths in u, times in s	
integrator	kick-drift-kick leapfrog at 1/60 s, bodies and craft together	symplectic for the bodies; second-order for the craft under thrust and drag
control tick	4 substeps = 1/15 s; the action held across the tick	craft and player act at 15 Hz; 2250 ticks per 150 s
gravitation	every massive body attracts every other	levels evolve; the unstable ones fail validation
softening	$GM_b\, r/r_b^3$ inside a body's radius r_b	finite everywhere (uniform sphere); zero self-force
the craft	a test particle: pulled by every body, moving none	body paths fixed by the snapshot: exact replay, and a browser ephemeris from 1.5 KB
capacity	32 body and 12 zone slots, masked; the pools use at most 12 bodies and 9 zones	one observation shape for every level
Craft and task	turn {left, none, right} at 3.5 rad/s × throttle {0, 0.3, 1}	nine actions, what a player can press; no angular inertia
action set		
engine	a_{eng} log-uniform on $[0.6, 3]$ u/s^2 at full throttle, per instance (suites override)	thrust comparable to local gravity; no craft can ignore the field
Δv budget	$\beta \in [0.55, 1.35]$ times the direct-flight estimate Δv_{req}, carried as $\beta\, \Delta v_{\text{req}}/a_{\text{eng}}$ s of full burn, clipped to $[3, 60]$ s	usually short of a straight flight, so gravity has to do part of the work
targets	1–6 per instance, at least 100 u apart: a point in free space, a station, or a point hung off a solid body at radius + atmosphere + 35 u	a moving target carries its anchor's velocity; black holes and tracers never anchor
order	slot order unless order-free (0 in training until the curriculum raises it, 0.3 on the blind suite)	the navigator chooses the sequence
arrival	within 12 u; rendezvous: also within 3 u/s of the target's velocity; flythrough: any speed	a rendezvous (moving targets only) is materially harder
Hazards, endings	$\rho_0 e^{-\text{alt}/(0.25\,h)}$ to thickness h; drag $0.05\,\rho\,\lVert v_{\text{rel}}\rVert\, v_{\text{rel}}$; heating $0.004\,\rho\,\lVert v_{\text{rel}}\rVert^3$ health/s	drag enters the dynamics, heating only the health
atmosphere		
radiation	$\sum_b \text{lum}_b/\max(r, r_b, 1\text{ u})^2$ health/s	a constraint independent of the body's gravity
zones	annuli, soft edge 15% of the thickness; debris 0.35× strength × speed relative to the flow; belt: flat strength; sensor: harmless	damage is continuous in position; matching a debris flow is nearly free
endings	per substep, in order: `crashed` (inside a solid body), `destroyed` (health, 100 at the start, reaches 0), `arrived` (every target visited), `lost` ($\lVert x\rVert > 1.35\,R$); per tick: `timeout`	exactly one status, `arrived` the only success; an empty tank is not one: burn is min(throttle · Δt, fuel left), the craft coasts on (Section 3.6)
Levels	12 procedural; 9 for training, 3 held out	whether a task was learned or a level memorised
families		
validation	to the horizon: no solid pair within radii + atmospheres + 3 u; orbits within 0.6–1.6 of their initial parent distance (0.3–3.0 for `rogue_flyby`); $\lvert\Delta E/E\rvert \le 2 \times 10^{-3}$	no collisions, no ejections, a trusted integration
snapshots	horizon 300 s (standard pools) or 900 s (long pools); state stored every 20 s, (horizon − clock)/20 s + 1 of them: 8 and 16	an episode starts at any epoch that leaves a full clock inside the validated window
level radius R	farthest reach of any body over the horizon + 100 u; 356–1876 u in the training pool	the escape boundary sits at $1.35\,R$
episode clock	150 s = 2250 ticks; `SPACENAV_EPISODE_S` sets it per process	ticks, the time-remaining entry and the snapshot count derive from it

Table 1: Modelling assumptions and fixed parameters of the simulation (`spacenav/constants.py`, `physics.py`, `env.py`, `levels/build.py`); the engine and budget ranges are overridden per evaluation suite (Table 3).

body dominates almost everywhere, so that a coarse one-body representation is nearly always adequate, and three (`binary_star`, `trinary`, `star_cluster`) are systems of comparable masses in which no single body dominates anywhere. The second is what, besides gravity, constrains the route, and the third is whether the dominant structure persists through an episode; the remaining families each introduce one feature along these lines, i.e. dense small bodies (`asteroid_field`), a tightly packed system (`compact_system`), extended hazard zones (`debris_maze`), a very massive compact object (`bh_cluster`), intense radiation (`pulsar`), and a single unbound body passing through (`rogue_flyby`). Generating levels procedurally across families, and testing on levels and families never trained on, is the usual means of measuring whether a policy has learned a task or only a level [23, 63, 64]. A generated level is accepted only if it remains well-behaved (no collisions, no ejections, bounded energy drift) for a validation horizon, and accepted levels are stored as snapshots at 20 s intervals so that an episode may begin at any of several epochs. Four pools are used throughout: 18,000 training levels, 2,700 validation levels from the same families, and 1,200 holdout levels from families never trained on, each validated to 300 s (twice an episode) and carrying eight snapshots; and 1,800 long-horizon levels validated to 900 s and carrying sixteen snapshots, for experiments at extended clocks (Section 3.6).

3.2 Nontrivial difficulty

The representation a person is most likely to bring to a gravitational field is the one-body representation, in which the dominant attractor is identified, its field is taken for the whole field, and trajectories are reasoned about as conic sections. This is instantiated in the environment by a function, `gravity_at(topk=k)`, that restricts the craft's felt gravity to the k strongest attractors at its current position and discards the rest, with $k = 1$ being patched conics and $k = 0$ the full field; bodies are always advanced under the full field, and only the test particle's felt force is restricted, so that a plan made inside the coarse representation can be flown in the true field and the discrepancy observed.

With a_{full} the true local acceleration and a_{dom} the acceleration due to the single strongest body, the fraction of the field that a one-body representation throws away is

$$\eta \;=\; \frac{\|a_{\text{full}} - a_{\text{dom}}\|}{\|a_{\text{full}}\|}, \tag{1}$$

which is near zero where patched conics is adequate and near one where it is not. However, η alone is not difficulty, because a craft with a large enough engine can absorb the error. With Δv_{budget} the Δv budget, $\|g\|$ the local gravitational acceleration and T the remaining flight time, the authority

$$A \;=\; \frac{\Delta v_{\text{budget}}}{\|g\|\,T} \tag{2}$$

is the budget in units of the impulse gravity itself will deliver over the flight ($A > 1$ meaning the craft can out-push the field, $A < 1$ that it cannot), and the quantity used to score a level is their ratio,

$$\mathcal{D} \;=\; \frac{\eta}{A}, \tag{3}$$

which has the direct reading of the share of the budget that flying the wrong representation would cost. Both ingredients are available in closed form from the level snapshot and the task specification, so $\mathcal{D}$ requires no simulation and thousands of candidate levels can be scored at negligible cost, with expensive planning reserved for levels already known to sit where they are

Family	Regime	Why it is in the set	Median η
`sol_like`	hierarchical	one star, well-separated planets; the baseline in which a coarse model is right almost everywhere	0.0027
`jovian`	hierarchical	a giant with a retinue of moons; dominance passes between bodies within a small region	0.0080
`saturnian`	hierarchical	as `jovian`, with a ring system that adds a hazard band	0.0058
`compact_system`	hierarchical	many bodies packed close; short orbital periods relative to the episode	0.0017
`asteroid_field`	hierarchical	many small bodies; collision risk without a change in the dominant field	0.0013
`debris_maze`	hierarchical	extended debris zones; the hazard shapes the route more than the gravity does	0.0005
`bh_cluster`	intermediate	a compact massive object among lighter bodies; a steep field with a contested shell	0.068
`binary_star`	contested	two comparable masses; no point at which one body dominates	0.35
`trinary`	contested	three comparable masses	0.47†
`star_cluster`	contested	several comparable masses; contested everywhere	0.53
`pulsar`	hierarchical	a luminous body whose radiation is the constraint and whose gravity is not	0.0026†
`rogue_flyby`	transient	one unbound body crossing the level; the dominant field changes during the episode	0.061†

Table 2: The twelve level families. Median η (the fraction of local gravity a one-body model discards, Section 3.2) over the playable area of each family's levels, computed by `scripts/paper_figures.py` from the long-horizon pool. †The three holdout families are absent from that pool and were measured on the holdout pool at its first snapshot ($n = 30$ levels each); `rogue_flyby` is transient, with the unbound body 1300–1600 u out at that epoch, so its η at later epochs is higher.

wanted. Moreover, it inverts: solving for the budget gives $\Delta v_{\text{budget}} = \eta \|g\| T/\mathcal{D}$, so that a level can be placed at a chosen $\mathcal{D}$ by solving for the budget (fuel is only ever added when retuning, since a level starved into a high $\mathcal{D}$ is infeasible rather than demanding).

An ordinary hierarchical level in this environment has η of order 0.003 over its playable area (Table 2) and a median A near 2.3 over sampled task instances ($n = 4{,}096$ draws from the training pool), so that $\mathcal{D}$ is of order 0.001: gravity supplies a substantial fraction of the budget's worth of impulse over a flight, but a single body explains more than 99% of the field, so that patched conics is adequate on an ordinary level and η has to be raised by construction. That $\mathcal{D}$ measures what it is meant to was checked by comparing a planner restricted to the one-body representation against one with the true field on levels pooled into two groups of $\mathcal{D}$ bands (24 levels per group): below $\mathcal{D} = 0.1$ the two were indistinguishable (32% versus 42% arrival, Fisher's exact $p = 0.74$), while above it they separated (13% versus 54%, $p = 0.005$), with the coarse planner failing every level in the highest band. Note that the figures for the individual bands within those groups, at $n = 8$ each, are not individually significant, and only the pooled comparison should be read.

$\mathcal{D}$ is a first-order measure, and its construction implies a particular kind of navigator, one that spends fuel to cancel every departure of the true field from its one-body model, i.e. one that treats the discarded part of the field as an error to be corrected. It prices the cost of flying a one-body model through a many-body field, and it credits nothing to a navigator that exploits the discarded part of the field instead, such as one that bets on the direction in which an encounter will throw it and spends fuel only to choose between outcomes, or one that lets an encounter or a long timescale supply the velocity change the budget could not pay for, which is what each design in Figure 1 does [32, 33]. A level that $\mathcal{D}$ scores as expensive can be cheap for such a navigator, and a level it scores as cheap can hide a scattering encounter that $\mathcal{D}$ does not see at all, since η is an instantaneous force ratio and says nothing about where trajectories diverge. In the model space of Section 1 these are the points at which a strategy that adds priors the one-body model lacks renders a level tractable that the one-body model alone cannot fly, and the remaining quantities of this section (the divergence rate of Section 3.4, the solvability probe of Section 3.5 and the clock of Section 3.6) are the beginnings of the higher-order terms such a navigator would need. What $\mathcal{D}$ does is to locate, cheaply and in closed form, the levels on which the one-body representation is insufficient and some richer one is required (Section 5.5 qualifies this for a learner at the present training budget).

Where in a level a coarse representation breaks is governed by instantaneous force competition rather than by the field's resonance structure, although the latter is the candidate the literature on constructed chaos suggests [65–67], since resonance overlap describes what happens over many orbital periods while an episode here is a fraction of one, and placing targets in the Wisdom band raises η only from 0.0026 to 0.0172, sixfold and still negligible, because a few Hill radii from a secondary has $\eta \approx 0.02$ throughout. The relevant surface for an episode this short is the one on which a secondary's pull equals its primary's, at a distance $d = a\sqrt{m/M}$ from a secondary of mass m orbiting a primary of mass M at semi-major axis a (about half a Hill radius, and distinct from the Laplace sphere of influence used in patched-conic design [4]). Figure 3(a) shows η measured on shells around the second-most-massive body of 297 levels spanning nine families, as a median over 24 directions at each radius: it rises from 0.09 at 0.3× the equal-pull radius to a peak of 0.43 at 1.0× and falls to 0.24, 0.066 and 0.020 at 2×, 4× and 8×. The peak value depends strongly on how it is sampled, since along the line joining primary and secondary the two pulls oppose and η exceeds one; the angular median is the conservative statistic and the one the level generator is tuned against, with target placement in contested levels confined to the 0.8–1.3× shell. The larger lever by far is family choice (Figure 3(b)): median η over the playable area spans three orders of magnitude from `debris_maze` (0.0005) to `star_cluster` (0.53), so that systems of comparable masses are contested everywhere and hierarchical systems only within thin shells that must be aimed at deliberately.

Figure 4 makes these quantities visible on four real levels, one per family (chosen as the level whose median η is nearest its family's median, so that none was chosen on its appearance), as fields over the playable area: η; a finite-time Lyapunov exponent over 20 s, measured by displacing a coasting particle and integrating the separation against the true ephemeris; and the single-burn outcome map, which records, for the level's start, which burn headings and burn times reach the target. The η row separates the families at a glance, since a hierarchical level is dark everywhere except in thin rings around its planets while a contested level is bright everywhere and the equal-pull surface is a visible dividing curve through the middle of it. The outcome row does not separate them, because a single-burn decision space is shaped by the task (the start orbit, the target's motion, the authority A) at least as much as by the family, so that the star cluster shown has a clean diagonal winning stripe (the signature of leading a moving target, since the right

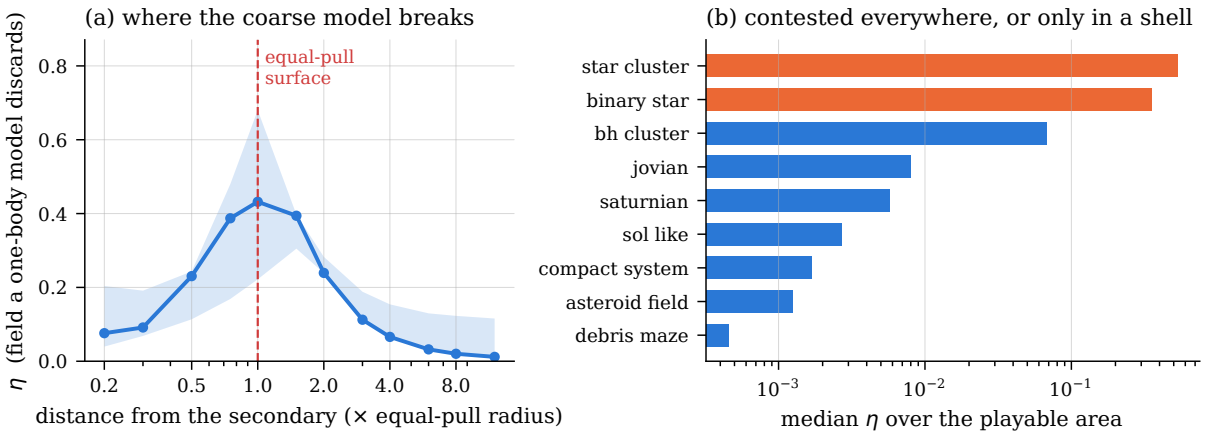


Figure 3: (a) The fraction η of local gravity that a one-body representation discards, against distance from a secondary in units of the equal-pull radius $a\sqrt{m/M}$; median over 24 directions on each shell, across 297 levels from nine families, with the interquartile range shaded. (b) Median η over the playable area by family, with the two families that are contested everywhere in orange. Computed by `scripts/paper_figures.py`.

heading advances with the burn time) and the binary star a single narrow band, while two other levels from the same families would show different shapes. A claim about the decision space of a family, as opposed to a task, needs many tasks per family and is not made here.

3.3 Trivial difficulty

Separate from whether a representation suffices is whether a correct plan can be executed, i.e. whether the window in which a decisive action must be taken is wide enough for the hand that must take it. Measured human limits (roughly 20 ms of motor onset jitter for a practiced discrete action [68], 200–250 ms of simple visual reaction time [69], and visuomotor feedback responses beginning 120–180 ms after a perturbation with a closed-loop correction ceiling near 3 Hz [70, 71]) give the scale, and since the 66.7 ms control tick is longer than the onset jitter of a practiced hand, the tick sets the floor on the time resolution of any action here, for policy and person alike, before the hand does. Every level is banded on this axis as well as on $\mathcal{D}$, and the two are never combined (Section 2). This kind of difficulty is called trivial here because it says nothing about the field, and it is not treated further.

3.4 Chaos on episode timescales

The first picture of Section 1 presupposes that the field is chaotic on the timescale of an episode, and the threshold at which chaos matters to a learner is sharp enough that the field has to be checked against it. Wang, Herbert and Gao [37] show that the reinforcement-learning objective is α-Hölder with $\alpha = \log(1/\gamma)/\lambda$ for a system with maximal Lyapunov exponent λ and discount γ, becoming fractal, and losing any usable descent direction, when λ exceeds $\log(1/\gamma)/\Delta t$; at the discount of $\gamma = 0.999$ and the 1/15 s control interval used here that threshold is $\lambda_{\mathrm{crit}} = 0.0150\,\mathrm{s}^{-1}$, corresponding to a Lyapunov time of 67 s. The measured predictability horizon of the environment (the time at which one control tick's worth of thrust error grows past the 12 u arrival tolerance) is 52–74 s across eight families, close enough to that figure that bounding the exponent is not sufficient and it has to be measured.

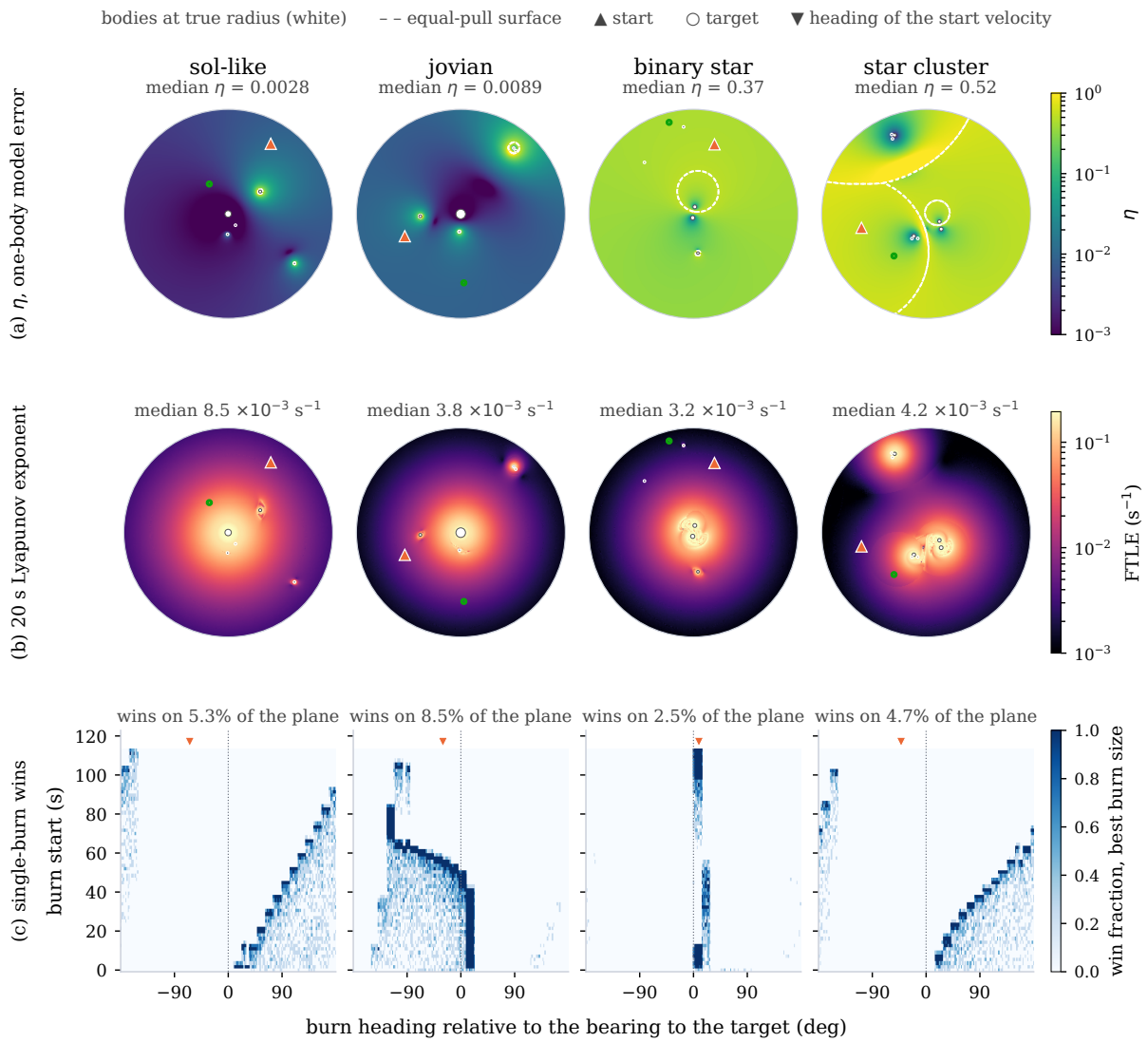


Figure 4: Computed portraits of four levels, one per family, with families as columns. (a) The η field over the playable area on a logarithmic scale, with bodies at true radius and the measured dominance boundary (where the strongest attractor changes) dashed; the start and target are marked. (b) A 20 s finite-time Lyapunov exponent from a displaced coasting particle, integrated against the true ephemeris. (c) The single-burn outcome map for the level's own task: the fraction of jittered draws that arrive, over burn start and burn heading relative to the bearing to the target, taking the best of four burn sizes. One color scale per row. Row (c) is the decision space of one task and should not be read as a property of its family. Computed by `scripts/fig_phase.py` on 368,640 full-episode rollouts per level.

The divergence is ballistic, since $d\ln(\text{sep})/dt$, which falls as $1/t$ for linear growth and stays flat for exponential growth [72, 73], decays in the measured separations in the manner of linear growth: pure linear growth predicts a 150 s-to-75 s slope ratio of exactly 0.50, and the measured ratio is 0.33–0.64 with a median near 0.56 (16 levels in each of eight families, one tick of thrust as the perturbation). Fitting $\text{sep} = a\,t\,e^{\lambda t}$ to divide out the ballistic factor gives $\lambda = 0.0045$–$0.0086\,\text{s}^{-1}$ (90th percentile near 0.010), roughly a factor of two below λ_{crit}, with only the tail of `star_cluster` exceeding it. A Lyapunov time of about 140 s against a 150 s episode is one e-folding per episode, or about 2000 control steps per e-folding, whereas the reinforcement-learning results on chaotic systems in the literature sit at 10–100 steps per e-folding [74–76], and the classical chaos-control results treat that sensitivity as the thing that makes small control effective [77, 78]; the environment is well inside the benign regime. The uncertainty exponent of the arrival basin [26, 79, 80], which is near one for a smooth boundary and near zero for a fractal one, measures 0.53–1.02 once enough perturbation samples are used to fit it (768 per level; with 96 the flip counts are small integers and no exponent can be trusted), consistent with the Lyapunov result. Thus, the field is quasi-integrable on the timescale of an episode, in the sense in which a near-integrable Hamiltonian system retains most of its invariant tori under perturbation and drifts only over times exponentially longer than its orbital period [81–83], and in the same sense in which the solar system is chaotic with a Lyapunov time of millions of orbits [84, 85], and where chaos is wanted, as it is for the first picture of Section 1, it has to be built into a level, chiefly by episodes long relative to orbital periods (closer encounters and tighter binaries are expected to help less).

3.5 Solvability

A closed-form $\mathcal{D}$ says how much a wrong representation would cost but not whether a level can be flown at all, and for that the most direct evidence is an attempt with the strongest navigator available. Here that is a cross-entropy-method trajectory optimizer [86], of the sampling-based kind that runs well on a graphics processor [87]: piecewise-constant control segments over the episode (48 segments of 3.1 s at the 150 s clock), seeded from a scripted reflex controller and from an explicit zero-thrust candidate, iteratively refined toward an arrival objective over 96 samples and 7 iterations, with the zero-thrust seed included because throttle is parameterized through a sigmoid and an all-coast plan is otherwise never drawn, which caused the optimizer to fail on the levels where an all-coast plan is the best one found. A level is called solvable if the optimizer arrives or if any member of an enumerated one- or two-burn family of control sequences arrives, and a level that nothing tried can fly receives no difficulty label, which is a design decision and should be read as one, and it is the nearest relative in this paper of the observation that levels an agent always or never solves carry no curriculum information [43, 44].

Two properties of this yardstick bear on every use of it. Firstly, its failure is weak evidence, since a given optimizer configuration flies a given level or does not, and a run of failures over a small set of levels is consistent with a true success rate well above zero (zero successes in twelve attempts is consistent with a rate of 16%, the rate observed on the same level family at $n = 32$, at probability 0.13), so that a level class should be called infeasible only on a sample sized to the claim [88, 89]. Secondly, and more fundamentally, the true optimum for a level is never in hand, because the optimizer is a sampled search over a restricted control family on a problem that Section 2 argues cannot be solved exhaustively, so that every difficulty label in this paper is relative to a navigator that is itself imperfect in ways that cannot be fully characterized, and a level labelled unflyable is one that nothing tried could fly. More computation narrows the gap without closing it, since the restricted control family and the sampled search remain, and that

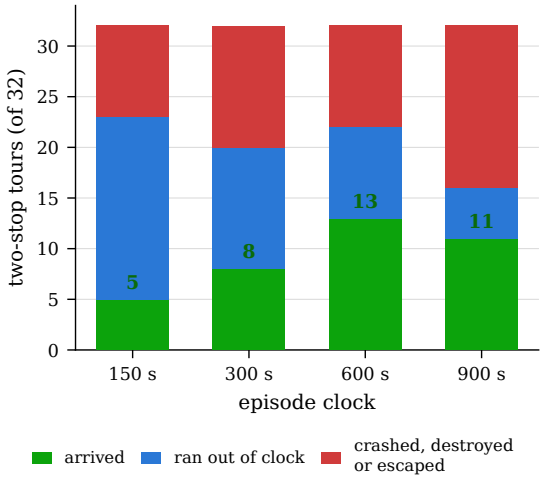


Figure 5: Thirty-two two-target levels flown by the reference optimizer at four episode clocks, control resolution held at 3.1 s per segment. Timeouts fall monotonically, arrivals peak near 600 s, and hazard endings rise.

is why the paper treats a learning agent (Section 4) as a second, independent probe of the same levels as well as the thing being graded.

3.6 The episode clock

Of the constraints a level places on a craft (time, fuel, and exposure to damage), time is the least interesting, and it is also the one most easily mistaken for difficulty. This is a position more than a finding, since deadlines and launch windows are real constraints in the trajectory-design sources cited above, but it is the position the measurement below supports. Fuel is a harder constraint, because it is consumed and cannot be recovered, and exposure is the most interesting, because radiation, debris and atmosphere each have their own geometry and dynamics and because avoiding them shapes a route in ways the gravity alone would not. The clock, by contrast, mostly penalizes a craft that chooses an energy-efficient path over a fast one, which is the choice every mission in Section 1 was judged on.

The evidence that the 150 s clock of this environment is binding in that way comes from the optimizer of Section 3.5 flown on 32 two-target levels at four clocks, with control resolution held at 3.1 s per segment so that a longer clock buys more segments rather than coarser ones (Figure 5). Arrivals rise from 5/32 at 150 s to 8/32 at 300 s and 13/32 at 600 s, then fall back to 11/32 at 900 s; timeouts fall monotonically from 18 to 5; and crashes, destructions and escapes go from 9 at the shortest clock to 16 at the longest, though not monotonically, so that of the thirteen timeouts a longer clock removes, about half become arrivals and half become hazard endings. Thus, for the optimizer at 150 s, a substantial fraction of what the environment scores as failure is the clock, and the useful ceiling for this task class appears to be near 600 s, beyond which hazard exposure is the binding constraint.

The same experiment on a weaker navigator gives the opposite reading, which is what makes the pair of measurements informative about the navigator and not only about the clock. Evaluation suites were rebuilt from a 900 s-validated pool at a 600 s clock (same families and task specifications as the 150 s suites, different levels, so same-family rather than paired), and the

scripted reflex pilot was flown on them: its arrivals after the 150 s mark are 1/480, 3/197, 1/47 and 0/136 on the easy, medium, hard and blind suites, and its 150 s timeouts (102, 146 and 147 of about 600 on the medium, hard and blind suites) became escapes and collisions at 600 s rather than arrivals (medium suite: timeouts 102 to 17, escapes 53 to 105, arrivals 206 to 197), an escape being the craft leaving the level past 1.35 times its radius, which a pilot with an empty tank cannot avoid once it is on an unbound path. A longer clock, that is, gives a navigator the time to finish a path it could not otherwise finish, or the time to spend its fuel and drift out, and which of the two it gets depends on the navigator.

The learned policy, trained at a 600 s clock, is a third case, and the one that argues most strongly against treating the clock as a free parameter. Three seeds of the best 150 s configuration were trained for 3000 updates with the clock at 600 s on the long-horizon pool and evaluated on the 600 s suites (same families and task specifications as the 150 s suites, different levels, so same-family rather than paired; and the blind suite is not blind at this clock, since no long-horizon pool carries the holdout families). Two seeds learned and one did not: the two that learned reached 68% and 74% on the easy suite and 26% and 32% on the medium suite, against 90% and 42% for the same configuration at 150 s, while the third never exceeded 19% on the easy suite and finished at 4%, so that the mean over seeds is 48.9 ± 38.6, 19.8 ± 16.1, 3.9 ± 3.6 and 13.1 ± 10.5 on the four tiers. Note that the dominant ending for the policies that did learn was still the clock: 37–42% of medium-suite tasks timed out at 600 s, against roughly a third at 150 s, so that four times the time produced more timeouts, not fewer. The likely explanation is the discount, since $\gamma = 0.999$ gives the value estimate a horizon of about 65 s at 15 Hz, which is a reasonable fraction of a 150 s episode and a ninth of a 600 s one, so that a policy trained at the longer clock is rewarded for progress it cannot see the end of and is not penalized for drifting. Raising the discount to match the clock confirms this, and the threshold is sharp. At $\gamma = 0.9995$ (about 133 s of lookahead) the failure reproduces almost exactly, with the same seed never learning and means of 47.7 ± 38.1, 20.0 ± 16.2, 4.2 ± 4.1 and 13.1 ± 11.1; at $\gamma = 0.9998$ (about 333 s, over half the episode) all three seeds learn, the seed standard deviations fall to 0.1–1.4 points, and the policy reaches 83.7 ± 1.4, 42.4 ± 1.4, 14.1 ± 0.7 and 29.7 ± 0.1 on the four tiers, i.e. at or above the 150 s reference on medium, hard and blind (on different levels, so same-family rather than paired, and with the caveat that the 600 s blind suite is seen-family) and somewhat below it on easy, with the medium-suite timeout share down from 37–42% to 26–35%. Thus, the clock and the discount behave as a single setting, since a longer clock is available to a learner only together with a horizon that reaches a substantial fraction of it, and the sense in which 150 s was arbitrary (Section 6) does not make 600 s correct so much as it makes the pair the quantity that has to be chosen.

The clock cannot simply be set long enough to settle the question, for a reason that bears on what solvability means. Consider a navigator that does nothing at all. The bodies and a coasting craft together form an autonomous Hamiltonian system, and on a bounded energy surface, i.e. one of finite phase-space volume, such a system is recurrent in the sense Poincaré established for exactly this problem [1]: almost every trajectory returns arbitrarily close to its starting state, and so, given enough time, a coasting craft will pass within the arrival radius of any target it can reach at all, unless an invariant surface of the kind that survives perturbation in a near-integrable system [81, 82] happens to separate it from that target permanently. Thus, for a flythrough tour, the zero-action policy succeeds with probability approaching one as the clock lengthens, and the only stopping criterion that is not a choice is divergence, i.e. the craft leaving the bounded region; inside it, there is always an improbable path that has not yet been given time to occur, and in the extreme of a system that collapses gravitationally toward a point, every path is that path.

A finite clock therefore belongs to the definition of the task, and solvability is relative to a time budget in the same way that Section 3.2 makes it relative to a fuel budget, and the question becomes how much a given navigator's success depends on the allowance. The same fact is what makes the exhaustive check of Section 2 ill-posed even in principle, since without a horizon the question of whether a control sequence has failed has no finite answer for a sequence that stays in the system.

The zero-action navigator was therefore measured directly (Figure 6). Coasting from the start of every task in the 600 s suites for 900 s (13,500 ticks, the levels' validated horizon), the fraction of tasks in which the craft passes within the arrival radius of any target (velocity tolerance and tour order ignored) is 2.3%, 2.9%, 5.1% and 3.7% at 150 s on the easy, medium, hard and blind suites, and 5.3%, 6.6%, 9.8% and 6.9% at 900 s, still rising at the end at about 0.2–0.5 percentage points per 100 s; strict completion of the tour as specified by coasting is 0–1.2% on every tier except easy (5.3%, where contact is completion). Over the same interval the fraction of tasks ending in collision, destruction or escape rises at about 2.5 points per 100 s on the three harder tiers, to 26–27% at 900 s. To see where the contact curve goes, a pool of 900 levels was validated to 9,000 s (sixty episodes; 80–100% of generated levels survive that long, so that the generators' levels remain bounded for at least that long) and the same measurement run on it: contact reaches 8.7%, 9.8%, 12.9% and 11.8% by 9,000 s and is flat over the last 300 s, because the craft is dead, with 78–80% of the non-easy tasks ended by then and destruction by accumulated damage the commonest ending. Thus, patience alone buys a coasting craft three to five points of contact between 150 s and 900 s and another three to six between 900 s and 9,000 s, while over the same spans the hazard endings rise faster, so that in this environment the stopping criterion that recurrence fails to supply is supplied by exposure.

4 The learning agent

A trained agent is used here both as a subject, evaluated on the levels as any agent would be, and as an instrument, since where and how it fails is a measurement of the levels that no closed-form quantity provides. The second use is the more important one for the argument, and it requires that the training procedure be ordinary enough that its failures are attributable to the levels rather than to the procedure.

The policy is an actor-critic over the ego-centric observation of Appendix A: a fixed self block, one token per body and one per hazard zone, with the tokens embedded and pooled two ways (multi-head cross-attention with the craft's own features as the query, and a masked max-pool), so that nothing depends on the number or order of bodies and the same weights run on any level. The network has an embedding width of 128, four attention heads and 285,322 parameters, which is small by the standard of current general-purpose agents [90]. Training uses proximal policy optimization [36] with generalized advantage estimation [91] in a fully vectorized loop of the kind that accelerator-native physics environments make practical [92] (1024 parallel environments by 128 steps, i.e. 131,072 transitions per update; four epochs of eight minibatches; learning rate 3×10^{-4} annealed linearly to a tenth; $\gamma = 0.999$; the advantage-estimation weight $\lambda_{\mathrm{GAE}} = 0.95$; clip 0.2; entropy coefficient annealed from 0.01 to 0.002; value coefficient 0.5; gradient norm clipped at 0.5), for 3000 updates or 393 million environment steps per run, which on the hardware used takes about 67 minutes. A curriculum [40, 41] ramps the share of rendezvous targets ($0.1 \to 0.45$), the maximum tour length ($1 \to 3$) and, after the craft can fly at all, the share of levels whose targets are placed in the contested equal-pull shell ($0 \to 0.35$), while the budget and engine start generous and tighten to their full ranges.

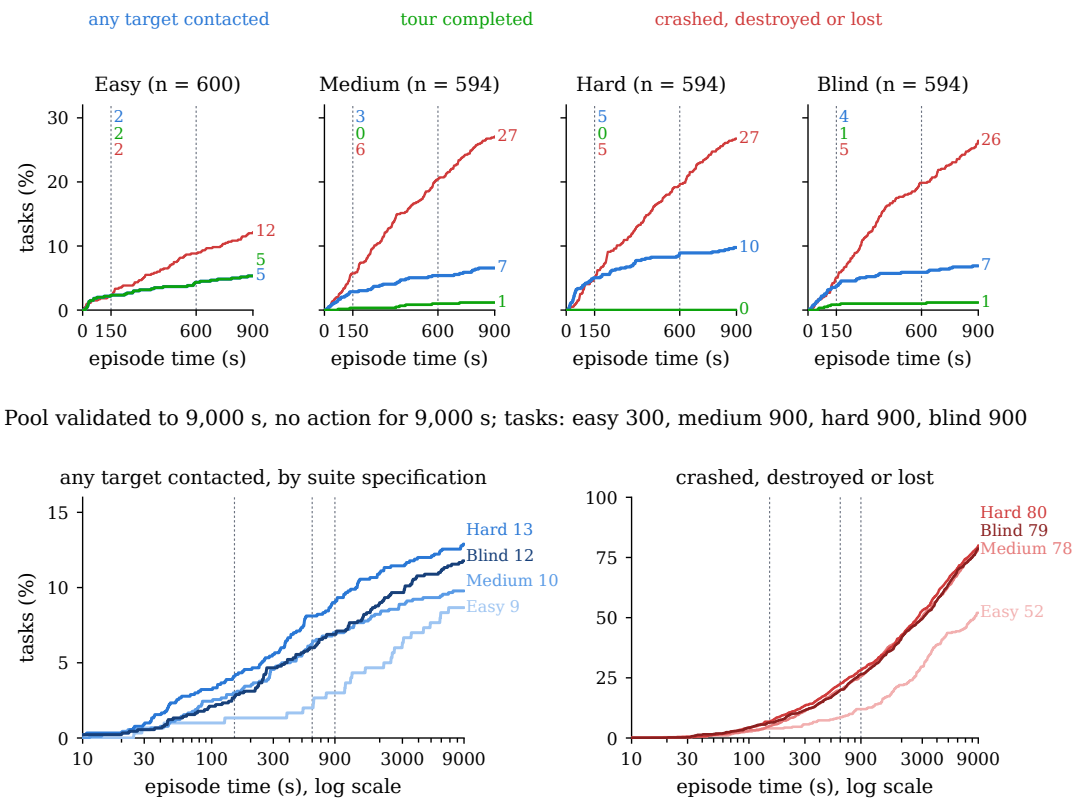


Figure 6: A craft that never thrusts. Top: on the four 600 s suites run to 900 s, the cumulative fraction of tasks in which the craft has passed within the arrival radius of any target, completed the tour as specified, or ended in collision, destruction or escape, with rules at the 150 s clock and at 600 s and the values at 150 and 900 s marked. Bottom: on 900 levels validated to 9,000 s, the same contact and death curves on a logarithmic time axis, by suite specification. Computed by `scripts/noop_clock.py`; the two rows are same-specification, not paired.

Evaluation is on four frozen suites of about 600 task instances each, described in Table 3 and held fixed across every run, with a scripted reflex controller (proportional navigation with gravity compensation, hereafter the reference pilot) scored on the identical instances as a yardstick. Six hundred instances give roughly two percentage points of binomial noise on a suite success rate, which is small against the differences reported below; the larger source of variance is the training seed, and every comparison between configurations is therefore made on seed means over five independent runs per configuration, following the statistical practice recommended for deep reinforcement learning [88, 89, 93].

5 Results

5.1 Performance against the reference pilot

Table 4 gives the final success rate of the best configuration found (the policy told η and the stretching rate, for reasons given below), as the mean of each run's last three checkpoints over

Suite	Pool	Targets	Rendezvous	Budget	Families	n	Pilot
easy	validation	1	none	1.0–1.5×	3 hierarchical	600	85.3%
medium	validation	1–2	25%	0.8–1.25×	all trained	594	34.7%
hard	validation	2–3	50%	0.7–1.05×	all trained	594	7.6%
blind	holdout	1–3	35%	0.75–1.15×	never trained	600	31.8%

Table 3: The evaluation suites. Budget is the Δv budget as a multiple of the direct-flight Δv estimate; the hard suite's budget is usually insufficient for a direct flight, so that gravity must do some of the work. Rendezvous is the probability that a target is a rendezvous given that it moves (a station or a target carried by a body, drawn with probability 0.45–0.6 per target), so the unconditional share is roughly half the column. The blind suite's tours are order-free 30% of the time. Pilot is the reference pilot's success rate on the identical instances.

	Easy	Medium	Hard	Blind
Learned policy (5 seeds)	90.4 ± 1.8	41.9 ± 2.3	11.1 ± 1.0	29.2 ± 1.2
Reference pilot	85.3	34.7	7.6	31.8
Difference	+5.1	+7.2	+3.5	−2.6

Table 4: Percentage of task instances completed, mean ± standard deviation over five training seeds for the policy told η and the stretching rate (the `eta,stretch` configuration), against the reference pilot on the identical instances. The no-channel configuration is statistically indistinguishable from this one at this budget (Table 6); the figures of Sections 1 and 5.4 fly policies from each.

five seeds, against the reference pilot on the same instances. The learned policy exceeds the pilot on the three suites drawn from trained families, by 3.5 to 7.2 percentage points, and falls 2.6 points short of it on the holdout suite, which is the only suite in which the policy sees level families it was never trained on and the only one in which the tour order is sometimes the navigator's to choose. Note that the reference pilot is a weaker yardstick than its rates suggest, since on a comfortable-budget instance set it arrives 63.3% of the time and its dominant failure is collision (21.6% of instances) rather than escape (4.7%) or timeout (7.8%), so that every margin quoted against it is softened by an opponent that flies into bodies one time in five, though the absolute rates are unaffected.

5.2 Training budget

Every configuration compared in this paper was trained for 3000 updates (393 million environment steps), and the question of whether that budget is where the comparisons should be made was answered by training the best configuration for 10,000 updates (1.31 billion steps, about 3.5 hours per run on the hardware used) at two seeds. The gain is large on every tier (Table 5): +4.7, +15.3, +7.8 and +11.7 percentage points over the five-seed 3000-update mean on the easy, medium, hard and blind suites, with seed standard deviations of 0.1–2.5 points, and the medium-suite curve of the longer runs rises from 47.8% at update 3000 through 52.4% at 6000 to 57.2% at 10,000 (the mean of the last three checkpoints) without flattening. Thus, the agent of Section 4 is under-trained at 3000 updates by a margin larger than most of the effects reported in this paper, and the ablations of Section 5.3, which were run at that budget, are comparisons between under-trained policies; the direction of an effect measured there is likely to survive more training, as the budget runs show the same ordering across tiers, but its size should not be read as a property of the task. The 10,000-update result is the ceiling this paper can report, and the one against which the

	Easy	Medium	Hard	Blind
3000 updates (5 seeds)	90.4 ± 1.8	41.9 ± 2.3	11.1 ± 1.0	29.2 ± 1.2
10,000 updates (2 seeds)	95.1 ± 0.9	57.2 ± 2.5	18.9 ± 0.6	40.9 ± 0.1
Difference	+4.7	+15.3	+7.8	+11.7
Reference pilot	85.3	34.7	7.6	31.8

Table 5: Percentage of task instances completed by the `eta,stretch` configuration at the two training budgets, mean ± standard deviation over seeds, on the 150 s suites. The 10,000-update runs passed the 3000-update mean at their own update 3000 (92.7, 47.8, 13.4, 34.8) and kept rising.

reference pilot of Table 4 should be compared: the learned policy then exceeds the pilot by 9.8, 22.5, 11.3 and 9.1 points on the four tiers, including the holdout tier on which the 3000-update policy fell short of it.

5.3 Observation channels

Having defined η, A and $\mathcal{D}$ as the quantities that describe a level's difficulty, the natural expectation is that an agent told those quantities learns better, and four channels were added to the observation on that expectation: η; a log-compressed authority A; a log-compressed demand $\mathcal{D}$; and the local tidal stretching rate (the square root of the tidal gradient, i.e. how fast nearby trajectories are pulled apart). The channels can be zeroed individually or together with the observation shape unchanged, so that an ablated run stays comparable with a full one, and toggling them was verified not to perturb the dynamics.

Supplying all four channels costs the agent between 3.1 and 10.1 percentage points of final success depending on the tier (Table 6; Figure 7(a); Welch's t-test on seed means, $n = 5$ per arm, $p = 0.021$, 0.0088, 0.029 and 0.0017 on the easy, medium, hard and blind suites, with effect sizes between 1.8 and 3.0). The quoted uncertainties are training-seed standard deviations, which are the relevant ones; the episode-level binomial intervals are far narrower and would overstate the confidence.

The observation already carries the local gravity vector (hence $\|g\|$), the product of fuel and engine acceleration (hence Δv_{budget}), and the fraction of the episode remaining (hence T), so that $A = \Delta v_{\text{budget}}/(\|g\|T)$ is an algebraic combination of three quantities the agent already has, partly duplicating an existing ratio of remaining to required Δv, and $\mathcal{D} = \eta/A$ is redundant given η and A; by contrast η requires the dominant-body decomposition, which is not otherwise computable from the observation, and the stretching rate is not present in any form (it is also a good proxy for the quantity one would actually want, since on the four levels of Figure 4 its log–log correlation with the measured 20 s Lyapunov exponent is 0.96–0.98). Splitting the panel along that line puts the entire cost on the redundant half (Table 6; Figure 7(b)). Supplying only η and the stretching rate is statistically indistinguishable from supplying nothing ($p = 0.40$ to 0.90 across suites), supplying only A and $\mathcal{D}$ is indistinguishable from supplying all four ($p = 0.59$ to 0.81), and the contrast between the two halves is significant on every suite ($p \leq 0.0064$, effect sizes 2.4 to 3.6).

In this environment, at this network size and step budget, appending features that are algebraic combinations of what the observation already contains measurably degrades a policy, while appending features that carry physics not otherwise available does not, and there is no evidence that the informative channels help (η and the stretching rate are nominally above the no-channel arm on three suites and below it on one, none significantly). Two accounts of the mechanism

	Easy	Medium	Hard	Blind
No channels	89.4 ± 1.50	41.7 ± 3.04	10.7 ± 1.24	30.2 ± 2.27
η, stretch	90.4 ± 1.81	41.9 ± 2.30	11.1 ± 1.03	29.2 ± 1.23
A, $\mathcal{D}$	86.6 ± 1.22	33.2 ± 2.61	6.2 ± 1.68	22.7 ± 2.52
All four	86.3 ± 1.87	31.6 ± 5.25	5.8 ± 3.39	22.2 ± 2.99
p (none vs. η,stretch)	0.40	0.90	0.60	0.42
p (all vs. A,$\mathcal{D}$)	0.77	0.59	0.81	0.79
p (η,stretch vs. A,$\mathcal{D}$)	0.0064	0.0005	0.0011	0.0022

Table 6: Percentage of task instances completed, mean ± standard deviation over five training seeds per arm, 3000 updates each; Welch's t-test on seed means.

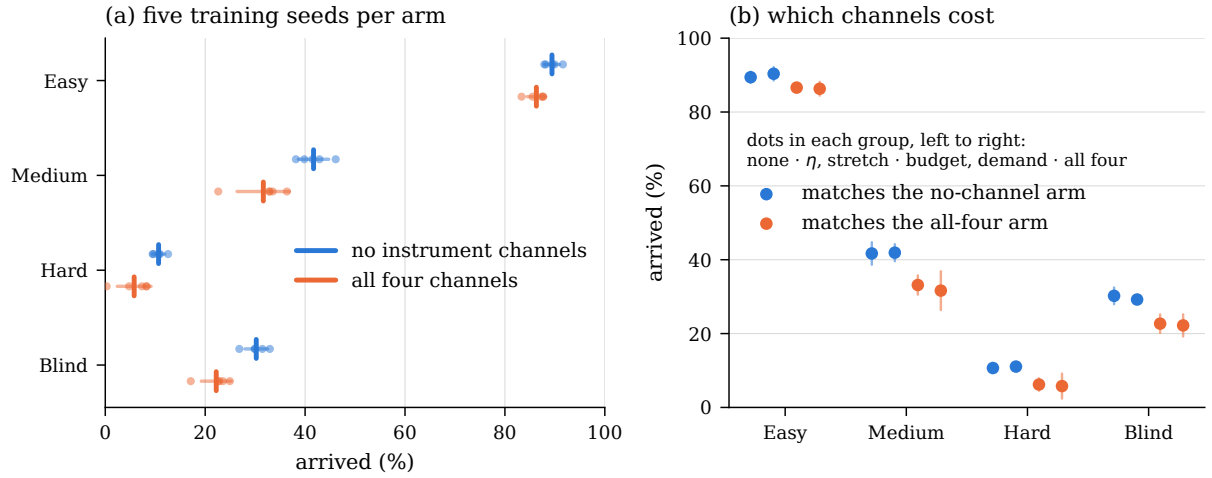


Figure 7: (a) Final success for every training seed, no channels against all four; dots are seeds, bars one standard deviation, the heavy tick the mean. (b) The four-way split; in every suite the no-channel and η,stretch arms coincide, and the A,$\mathcal{D}$ and all-four arms coincide below them.

predict different behavior under scale: four extra input dimensions against 285,322 parameters is a capacity account, under which a wider network should absorb the redundant channels, while a ratio with a division in it is heavy-tailed even after log compression, which is a conditioning account under which the cost should persist at any width. Doubling the embedding width to 256 (698,634 parameters, three seeds per arm, same budget) decides between them: the cost falls to -0.3, $+2.4$, $+0.8$ and $+2.7$ points on the four tiers (Welch's t, $p = 0.74$, 0.18, 0.38 and 0.026), the all-four arm at the wider network (90.1 ± 0.9, 41.2 ± 0.4, 11.4 ± 1.2, 29.6 ± 0.5) matches the η,stretch arm at either width, and the η,stretch arm itself gains little from the extra width (-0.6, $+1.7$, $+1.1$ and $+3.1$). The cost was capacity. Thus, an observation channel's value to a learner depends on whether it is new rather than on whether it is true, and at a network size where capacity is scarce a channel that does not change the policy costs capacity without buying anything, which is consistent with the view of a policy as a compression of its inputs [29, 94]; at a size where capacity is not scarce the redundant channels are merely ignored, and the wider network spends its extra parameters on tolerating them rather than on the task.

5.4 Per-instance variation

A suite success rate is one number per checkpoint, computed from a greedy policy that takes its most probable action, and it conceals both the policy's own stochasticity and the variation between individual task instances, which in procedurally generated environments is known to be large [23, 24, 88]. Figure 8 makes both visible by flying every saved checkpoint of one training run on one fixed instance sixteen times with actions sampled rather than maximized ($31 \times 16 = 496$ trajectories per instance), colored by the training update at which each attempt was flown. Early policies spray across the field and late policies converge into bundles, which are not always the right ones; on the hard-suite instance shown, the trained policy converges onto a tight shuttle between its first two targets and never reaches the third, in 0 of 496 attempts. Firstly, sampled success is far below greedy success (the easy-suite instance shown arrives 52% of the time when sampled against 89% for the greedy policy on that suite), so that the headline number describes a behavior the policy exhibits only with its stochasticity removed. Secondly, the spread between instances within a suite (60%, 52%, 8% and 0% on the four shown) is larger than any effect measured between training configurations in this paper, since the channel effect of Section 5.3 is three to ten points and two instances drawn from the same suite differ by fifty. Thus, any claim about this environment that rests on a handful of instances is fragile by construction (compare Section 3.5).

Figure 9 gives the corresponding learning curves, which show that mid-run checkpoints are noisy enough that the difference between two configurations read at a third of the way through training can have the opposite sign from the difference at the end.

5.5 Relations between the measures

The quantities of Section 3 were defined to describe a level's difficulty before anyone flies it, and the test of that is whether they predict who arrives. Across the 2,388 task instances of the four 150 s suites, with agent success per instance taken as the fraction of four 3000-update policies (two no-channel seeds and two all-channel seeds) that arrived at their final checkpoint, success falls monotonically with $\mathcal{D}$ (Figure 10(a); Spearman $\rho = -0.585$, $p = 10^{-219}$), from 84% in the lowest decile to 0.2% in the highest, with a plateau over $\mathcal{D} \approx 0.003$–$0.05$ and a cliff above $\mathcal{D} \approx 0.2$, and the reference pilot follows the same curve ($\rho = -0.547$; the two navigators' per-instance successes correlate at $\rho = 0.787$). A logistic model of agent success on family alone reaches a held-out area under the curve of 0.745; $\mathcal{D}$ alone reaches 0.811 and family with $\mathcal{D}$ reaches 0.871, and adding $\mathcal{D}$ to a model that already knows the suite, the family, the number of targets and whether a rendezvous is required still drops the held-out deviance by 1042 on one degree of freedom ($p = 10^{-228}$; ten-fold cross-validation repeated five times). Thus, $\mathcal{D}$ predicts success better than family alone and better than the suite's own design levers, as it was meant to.

However, it does so for the wrong half of itself. Separating the ratio into its parts, the authority A alone is the stronger predictor ($\rho = +0.754$; held-out area 0.946 with the design levers), while η at the targets is weak ($\rho = -0.248$) and, once A is in the model, carries no information at all (deviance drop 0.2 on one degree of freedom, $p = 0.67$; coefficient -0.02 against $+2.09$ for A). The relation between success and η is in fact non-monotone (Figure 10(b)): it peaks near $\eta \approx 0.005$ and falls on both sides, and the low-η failures are the hazard families (`debris_maze` 25%, `asteroid_field` 28%, `compact_system` 31%, `pulsar` 39%), where the field is simple and the route is not. What the learned policy's success tracks at this training budget is the fuel budget against the tour, i.e. whether a direct flight is affordable, and not whether a one-body representation of the field is wrong; the representational component of difficulty that the dial was built to capture

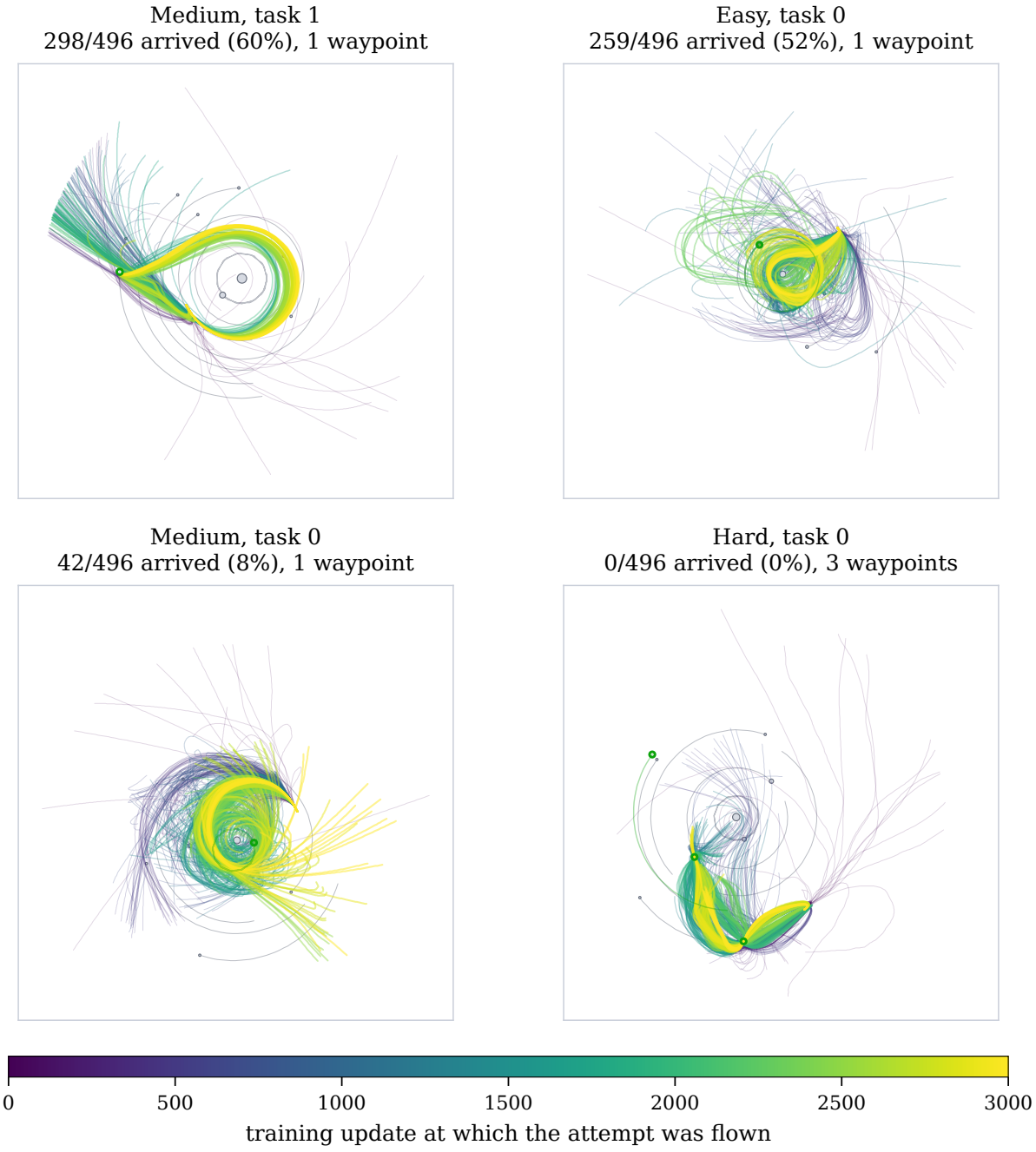


Figure 8: Four task instances from the evaluation suites, each flown 496 times by one training run of the `eta,stretch` configuration (every saved checkpoint, sixteen sampled rollouts each), colored by the training update at which the attempt was flown. Grey discs are bodies with their orbits traced faintly; green rings are targets, drawn along their own paths where attached to moving bodies. Early policies (dark) spray across the field; late policies (bright) converge into bundles. Medium task 1 converges onto a single arrival corridor; easy task 0 onto a corridor that arrives about half the time when sampled; medium task 0 onto a corridor that mostly misses; hard task 0 onto a shuttle between its first two targets that never reaches the third, at upper left.

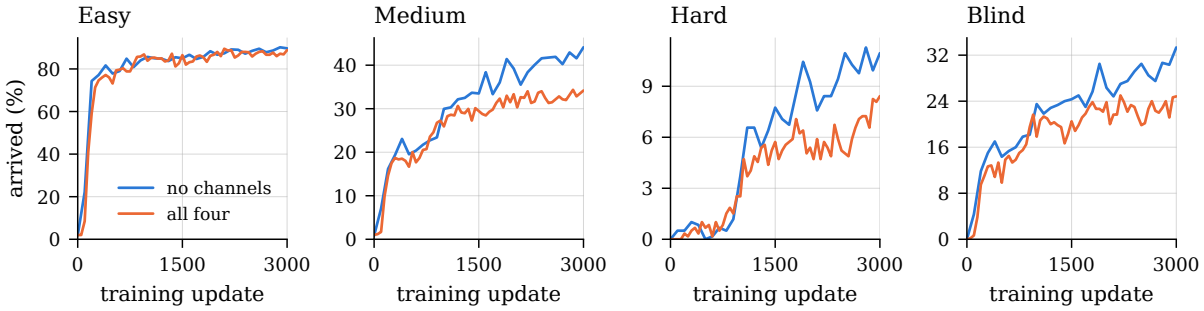


Figure 9: Suite success against training update for one seed of each headline configuration.

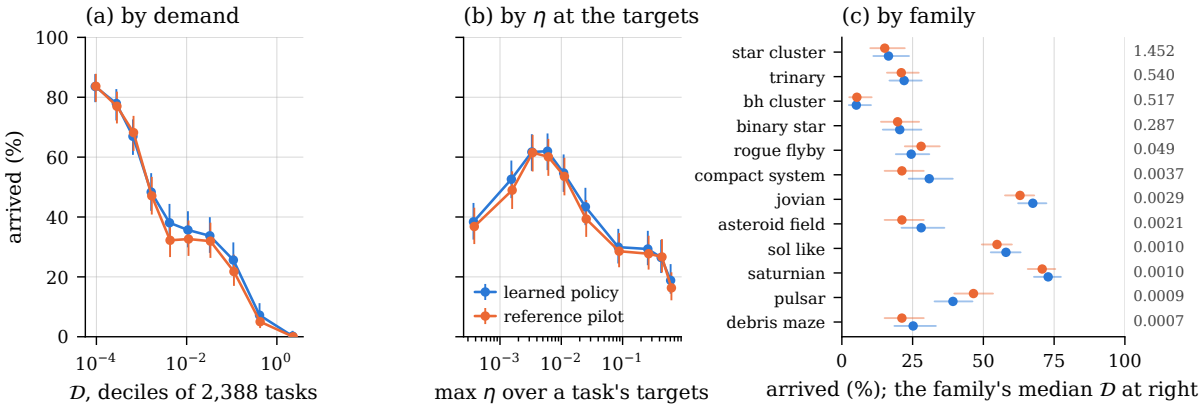


Figure 10: Per-instance success of the learned policy (mean of four 3000-update runs) and of the reference pilot across the 2,388 tasks of the 150 s suites. (a) By decile of $\mathcal{D}$, with 95% Wilson intervals. (b) By the maximum η over a task's targets; the relation is non-monotone, and the low-η failures are the hazard families. (c) By family, with each family's median $\mathcal{D}$ at right. Computed by `scripts/correlations.py`.

is not yet the component that limits the learner. This is consistent with the budget result of Section 5.2, in which the policy is still improving rapidly at 3000 updates, since a policy that has not yet learned to spend its budget well fails on fuel before it can fail on representation, and whether η becomes predictive once A is no longer the limit is a question the 10,000-update policies could answer and the present ones cannot. At the level of families (Figure 10(c)) the picture is the same from the other side: a family's median $\mathcal{D}$ predicts its success rate ($\rho = -0.734$ over the twelve families, $p = 0.007$), and the families that break the ordering are the ones whose difficulty is hazard rather than field.

The expectation that a longer clock matters most on multi-target tasks does not survive normalization. On the 600 s suites, a coasting craft first contacts a target after the 150 s mark on 2.3% of single-target and 4.7% of multi-target tasks (Fisher's exact $p = 0.001$), but per target slot the rates are 2.3% and 2.1% ($p = 0.81$), and for the first target alone 2.3% and 2.2%; more targets are more chances, and the tour structure adds nothing. Note that this measurement is on different levels from the 150 s suites and is not joined to the agent's per-instance success.

6 Discussion

Although the task examined here is fully specified and cheap to simulate, it does not admit a one-dimensional notion of difficulty, which is the first thing this paper is meant to establish and the reason Section 1 offers two pictures of it beyond the learning loop alone. A learner that treats the task as an undifferentiated observe-act-reward loop will learn it on levels where a coarse representation suffices and will have no particular reason to do better on levels where it does not, and the results of Section 5 are consistent with that account only in part: the policy is strong on the hierarchical families whose difficulty is the field (`sol_like`, `jovian`, `saturnian`), weak on the contested ones, worst on the held-out families where the representation learned elsewhere does not transfer, and, as Section 5.5 shows, also weak on the hierarchical families whose difficulty is hazard rather than field, which is the part of the picture a representational account of difficulty does not reach.

6.1 Learning a class of levels

The pedagogical question that follows is how an agent should be brought through the model space of Section 1, from the simplest representation toward the richest, so that what it learns on one level is the representation it needs on the next rather than a policy fitted to that level's particular geometry, which is the question the literature on generalization across procedurally generated levels and on learning to learn across task distributions addresses from two directions [23, 24, 95–97]. The curriculum used here (rendezvous share, tour length, contested share, each ramped over training) is a crude version of this, since the contested share was held at zero until the agent could fly at all, which is to say that the agent learned a one-body representation first and was then asked to extend it, and whether that order helps or hinders is a question the present data cannot answer. The literature on choosing which instances to train on next [43, 98–104] offers tools, with the caveat that several of its curriculum signals correlate with an agent's success rate rather than with its regret, and so miss the instances an agent can sometimes solve but not reliably [44], which are the instances the model-space picture says are the informative ones. What the closed-form $\mathcal{D}$ offers to this problem is a curriculum axis that a generator can set directly, placing an instance at a chosen share-of-budget cost of a wrong representation, so that an agent can be walked up that axis instead of being shown harder levels in the hope that they are hard in the right way.

6.2 Learning over a chaotic simulation

The first picture of Section 1 asks for something the plain learning loop cannot provide, which is a policy that bounds its exit from a region it cannot plan through. The measurement in Section 3.4 says that the present environment does not yet contain such regions on episode timescales, which is a limitation of the environment more than of the picture, and the route to building them is known (Section 3.4). Once they exist, and since the Hölder result says a policy gradient cannot be computed usefully inside such a region, the relevant question is whether a policy can learn the bounding behavior at the region's entrance and the adaptive behavior at its exit while treating the interior as noise, which is closer to the robust and tube-based formulations of control [27, 28] than to anything in standard reinforcement learning, and closer still to the observation from flow control that the gradient through a chaotic system is correct and useless at once [38, 39, 105]. Whether the learned representation survives passage through such a region is the question that

would decide between a model-free and a model-based approach, and the present data do not reach it.

6.3 Choices in the definitions

Several of the definitions in this paper are choices, and the reader should know which. Firstly, the equal-pull surface is the right scale for an episode shorter than an orbit and the wrong one for a longer episode, where resonance overlap reasserts itself (Section 3.2). Secondly, the 150 s clock was inherited from an early version of the game and never derived from the task, and Section 3.6 argues it is too short. Thirdly, the banding of trivial difficulty against human motor limits is a reasonable scale for a game and an arbitrary one for any other purpose. Fourthly, the reference optimizer is one configuration of one method, and every solvability label is relative to it. Finally, the twelve families were chosen to span the regimes the author could think of, with no taxonomy of the field behind them. These choices do not make the measurements wrong; they mean that the paper offers definitions that make a set of intuitions checkable, not a theory of difficulty for this task, which is also the condition of the task-difficulty measures and curricula in the literature, where the measure chosen changes what the curriculum does [41, 42, 44]. The value of better definitions here is that they let the environment and the agents that run over it be understood together, with the same quantities describing what a level demands and what a learner has managed, and that is a more useful thing to have than either description alone.

7 Future directions

The most immediate extensions are the ones the measurements call for, which are a longer clock, levels with constructed chaotic regions (Section 3.4), and training budgets and seed counts large enough that the scaling behavior of the observation-channel result can be read. Beyond navigation itself, the same field and the same representational questions arise in adversarial situations, such as in a video game in which one craft may be attempting to catch, target, or otherwise react to another, where the moving target is itself a navigator [27, 43, 49].

Acknowledgements

The picture of a many-body field as seen from inside a trajectory, through reachable sets, zero-velocity curves and manifold transfers, is owed to the videos of the *braintruffle* channel [106–108], which were the visual inspiration for this environment. The environment, the experiments and the analysis reported here were developed in collaboration with Claude (Anthropic), operating as an autonomous agent with access to the repository and compute; the repository's decision records were written during that process.

A The observation vector

What the policy is given at each control tick is the output of `observe()` in `spacenav/env.py`, a dictionary of fixed-shape arrays rather than a single vector, so that a level with seven bodies and one with thirty-two produce the same shapes and can share a batch: a self block of 109 numbers; one token of 24 numbers for each of the 32 body slots, with a 32-entry mask marking which slots hold a body; and one token of 15 numbers for each of the 12 hazard-zone slots, with a 12-entry

mask, which is 1,057 numbers in all ($109 + 32 \times 24 + 12 \times 15$, as measured by running `observe()` on a task from the frozen medium suite). Everything is expressed in the craft's own frame, i.e. positions and velocities are taken relative to the craft and rotated so that its nose lies along $+x$, with two fixed scales, $L = 100\,\mathrm{u}$ for lengths and $V = 20\,\mathrm{u/s}$ for speeds, and a signed logarithmic compression $\mathrm{symlog}(x) = \mathrm{sign}(x)\ln(1 + |x|)$ applied wherever a quantity spans several orders of magnitude. Throughout the tables, a_{eng} is the task's engine acceleration, $a_{\max} = 4\,\mathrm{u/s^2}$ is the fixed upper end of the engine range (`SHIP_ACCEL`, used only as a normalizer), R is the level radius, $T_{\max}$ is the episode clock (150 s unless `SPACENAV_EPISODE_S` says otherwise, Appendix D), and "the target" is the waypoint currently being flown to, which in a fixed-order tour is the next unvisited one and in an order-free tour is the nearest unvisited one (Appendix C).

Entry	Formula or source	Dim.
Craft and field		
craft velocity	$\mathrm{rot}(v)/V$	2
local gravity	$\mathrm{rot}(g)/a_{\max}$, g the full field at the craft	2
The target		
relative position	$\mathrm{symlog}(\mathrm{rot}(x_t - x)/L)$	2
bearing	unit vector of $\mathrm{rot}(x_t - x)$	2
distance	$\mathrm{symlog}(\|x_t - x\|/L)$	1
relative velocity	$\mathrm{rot}(v_t - v)/V$	2
Loadout, tour and state		
fuel fraction	$\mathrm{fuel}/\mathrm{fuel}_0$, seconds of burn left over seconds carried	1
Δv remaining	$\mathrm{fuel} \cdot a_{\mathrm{eng}}/100$	1
Δv remaining over required	$\mathrm{fuel} \cdot a_{\mathrm{eng}}/(\Delta v_{\mathrm{req}} + 10^{-3})$	1
Δv required	$\mathrm{symlog}(\Delta v_{\mathrm{req}}/100)$, from `deltav_estimate` over the pending legs	1
engine	$a_{\mathrm{eng}}/a_{\max}$	1
health	health/100	1
time remaining	$1 - t/T_{\max}$	1
pending targets	number unvisited / 6	1
order-free flag	0 or 1	1
damage rate	symlog of the current damage rate (health/s)	1
exposure	soft-edged membership, 0–1, of the zone the exposure objective charges for	1
radial position	$\|x\|/R$	1
objective weights	time, path, fuel, exposure; sum to one	4
instrument panel	η, symlog A, symlog $\mathcal{D}$, 10×stretch, each times its 0/1 mask	4
Per waypoint slot, six slots, 13 each		
relative position	$\mathrm{symlog}(\mathrm{rot}(x_k - x)/L)$	2
bearing	unit vector of $\mathrm{rot}(x_k - x)$	2
distance	$\mathrm{symlog}(\|x_k - x\|/L)$	1
relative velocity	$\mathrm{rot}(v_k - v)/V$	2
arrival radius	r_k/L (always 0.12)	1
velocity tolerance	$\min(v_k^{\mathrm{tol}}, 50)/V$ (0.15 for a rendezvous, 2.5 for a flythrough)	1
four flags	visited; is the current target; active; anchored to a body (0/1 each)	4
total	$31 + 6 \times 13$	109

Table 7: The self block. x, v are the craft's position and velocity, x_t, v_t the current target's, x_k, v_k the k-th waypoint slot's (an inactive slot reports the position its sampler left in it, masked by the active flag), and rot the rotation into the craft's frame.

Entry	Formula or source	Dim.
relative position	$\mathrm{symlog}(\mathrm{rot}(x_b - x)/L)$	2
bearing	unit vector of $\mathrm{rot}(x_b - x)$	2
distance	$\mathrm{symlog}(d/L)$, $d = \lVert x_b - x \rVert$	1
relative velocity	$\mathrm{rot}(v_b - v)/V$	2
pull	$\mathrm{symlog}\big(GM_b/\max(d, r_b)^2/a_{\mathrm{eng}}\big)$, the body's acceleration on the craft in units of the engine	1
mass	$\ln(1 + GM_b)/10$	1
radius	r_b/L	1
atmosphere	thickness$/L$ (0 for none)	1
luminosity	$\ln(1 + \mathrm{lum}_b)/10$	1
clearance	$(d - r_b - \mathrm{thickness})/L$, signed	1
carries a target	1 if any active waypoint is anchored to this body	1
kind	one-hot over ten kinds (none, star, rocky, giant, moon, black hole, station, asteroid, tracer, neutron star)	10
total		24

Table 8: The body token, one per slot for 32 slots, with the mask `body_mask` equal to the level's `active` flags. x_b, v_b, GM_b and r_b are the body's position, velocity, gravitational parameter and radius. Stations and tracers are massless and have zero pull; a tracer is the invisible anchor of a moving zone.

The four instrument channels occupy the last four entries of the self block before the waypoint slots, in the order η, A, $\mathcal{D}$, stretch, and are computed by `instrument_panel()` at the craft's position from the same quantities the body of the paper defines: η is $\lVert g_{\mathrm{full}} - g_{\mathrm{dom}} \rVert / \lVert g_{\mathrm{full}} \rVert$ with g_{dom} the field restricted to the single strongest attractor (`gravity_at(topk=1)`); the authority A is $\mathrm{fuel} \cdot a_{\mathrm{eng}} / (\lVert g_{\mathrm{full}} \rVert \, T_{\mathrm{left}})$ with T_{left} the time to the clock, floored at one control tick, and is passed through symlog; the demand $\mathcal{D} = \eta/A$ is likewise passed through symlog; and the stretching rate is $\sqrt{\sum_b 2\,GM_b/r_b^3}$ over the active bodies, i.e. the square root of the summed tidal gradient (a rate in 1/s, with r_b floored at the body's radius or 1 u), multiplied by ten so that it sits in the same range as the other entries. Each channel is then multiplied by a 0 or 1 taken from the environment variable `SPACENAV_INSTRUMENTS`, which is read once when `spacenav.env` is imported and accepts, in order of specificity, `0` (or `off`, or the empty string) to zero all four, `1` (or `on`, `all`) to pass all four, a comma-separated subset of the names `eta`, `budget`, `demand`, `stretch` (the names the code gives η, A, $\mathcal{D}$ and the stretching rate; an unknown name raises), or a four-character mask of 0s and 1s in that order, with `eta,stretch` as the default (the `eta,stretch` arm of Table 6; the no-channel arm is `0` and the all-four arm `1`). Because a zeroed channel keeps its slot, the observation shape, and hence the network, is identical under every setting, and nothing checks that a checkpoint is evaluated under the setting it was trained under; a policy trained with a channel live and evaluated with it zeroed reads a zero where it learned to expect a value, and the converse reads a value where it learned to expect a zero, in both cases without an error and with a success rate that simply looks wrong. The setting a run was trained with is written to its `config.json` under `instruments`, and `SPACENAV_INSTRUMENTS` should be set to that string whenever the run's checkpoints are replayed or evaluated.

The body and zone tokens are each passed through a two-layer embedding to a width of 128 and concatenated into one set of 44 tokens, which the network pools in two ways at once, by multi-head cross-attention (four heads) in which the embedded self block is the query and the tokens are keys and values, with the masked slots given an attention logit of -10^9 so that they receive no weight, and by an element-wise maximum over the unmasked tokens' rectified embeddings;

Entry	Formula or source	Dim.
relative position of the center	$\mathrm{symlog}(\mathrm{rot}(x_z - x)/L)$, x_z the anchor body's position (the origin for an unanchored zone)	2
bearing	unit vector of $\mathrm{rot}(x_z - x)$	2
distance to the center	$\mathrm{symlog}(d_z/L)$	1
inner radius	r_{in}/L	1
outer radius	r_{out}/L	1
signed distance to the band	$\mathrm{symlog}\big(\max(r_{\mathrm{in}} - d_z,\ d_z - r_{\mathrm{out}})/L\big)$, negative inside	1
kind	one-hot over none, debris, radiation, sensor	4
strength	the zone's damage coefficient	1
spin	$+1/-1$ if the debris circulates around the anchor, 0 if it moves with it	1
counts as exposure	1 if this is the zone the task's exposure objective charges for	1
total		15

Table 9: The zone token, one per slot for 12 slots, with the mask `zone_mask` equal to the level's `zone_active` flags. A zone is an annulus between r_{in} and r_{out} around its anchor, with a soft edge of 15% of its thickness.

the self embedding and the two pooled summaries are concatenated and fed to the shared policy and value trunk. Neither pooling depends on the number of bodies or on their order in the slots, which is what allows the same weights to run on every level, and it is why a level's bodies can be stored and presented in whatever order its generator produced them.

B Theoretical notes on observation

The observation of Appendix A is exact and instantaneous: at every control tick the policy is handed the true position and velocity of every body and target as of that tick. Two departures from this are worth setting down, because each would change what a learner could and could not infer, and because the environment's construction makes one of them unusually clean to study.

The first is a finite signal speed. If the craft learned of a body's state by light, a body at distance r would be seen where it was a time r/c ago, and at game scale this is not negligible: with a signal speed chosen so that the far side of a 3000 u level is one control tick away ($c \approx 45{,}000$ u/s, or one level-crossing per 66.7 ms), a body at 300 u would be seen one tenth of a tick late and a body across the level a full tick late, so that the observation would be internally inconsistent, with near bodies reported more recently than far ones. A learner given such an observation faces a problem that the present one does not pose, which is that the state it is handed is not the state of any single instant, and the correction (extrapolating each body forward by its own delay) requires the body's velocity, which is itself delayed. Whether a policy of the size used here can learn that correction implicitly, and at what cost in success, is an experiment the environment supports with a change to `observe()` alone, since the delayed state is available from the stored ephemeris; it has not been run. The formal setting for it is the partially observable Markov decision process [29], in which the delayed observation defines a belief over the current state.

The second is the distinction between a projected observation and a measured one, and here the environment's design has a consequence that is easy to overlook. Because the craft is a test particle, the bodies' future is fixed by the level's initial snapshot, and a navigator that knows the snapshot can compute every body's position at every future tick exactly, which is what the

browser client does when it integrates the ephemeris itself from 1.5 KB of state. The observation the policy receives is therefore equivalent to a projection from the snapshot, and nothing in it is a measurement in the sense of a quantity that could have come out otherwise. In a real setting the two differ: a measured position carries sensor noise, a projected one carries model error that grows with the projection horizon, and a navigator must weigh one against the other. The present environment collapses the distinction by construction, which is a simplification to be aware of when reading the results, and which the instrument channels of Section 5.3 partly reintroduce, since η is exactly a measure of how wrong a one-body projection of the field would be. A version of the task in which the craft perturbs the bodies (so that the ephemeris cannot be precomputed) or in which body positions are observed with noise (so that projection and measurement disagree) would restore the distinction, and the second is cheap to add.

A third note concerns the action set rather than the observation. The nine discrete actions (Figure 2c) were chosen so that a person at a gamepad and the policy choose from the same set, and so that the policy gradient is taken over a categorical distribution, which is the simplest case. The alternatives are a continuous thrust vector (a heading and a magnitude at every tick), impulsive burns in which the decision is when and how hard to fire and the craft coasts between them, which is the parameterization of classical trajectory design and of the reference optimizer of Section 3.5, and a set of maneuver primitives (a transfer, a plane change, an assist) from which a policy selects, which is the options construction [109]. Each changes what a learner can express: a continuous optimum that lies between two discrete throttle levels can be approximated here only by alternating levels across ticks, and a burn timed to a fraction of a tick cannot be expressed at all, which is one of the reasons the control tick is also the floor of trivial difficulty (Section 3.3). None of the alternatives was run, and the environment supports the first with a change to the action decoder alone.

C Goal conditions

A task instance is a tour of between one and six waypoints, sampled one after another by `sample_tour()` in `spacenav/env.py` so that each lies at least 100 u (`min_leg`) from the one before it, and each is of one of the three kinds in Table 10, which differ in what the waypoint is attached to and therefore in how it moves. Whether a waypoint moves is decided first (with probability `p_station`, 0.35 by default and set per suite), and a moving waypoint is a station when the level has a free one and a coin comes up heads, otherwise a point hung off a solid body; a waypoint that does not move is a point sampled uniformly over the playable disc, rejected and redrawn (up to 96 candidates) until it lies between `min_dist` and `max_dist` from the previous waypoint, inside 0.9 of the level radius, at least 20 u clear of every surface and atmosphere, and at a point where the instantaneous damage rate is under 0.3 health/s. The velocity condition is drawn second and only for a moving waypoint, with probability `p_rendezvous` (so that the rendezvous shares quoted in Table 3 are conditional on the waypoint moving, and a fixed point in free space is always a flythrough). Every waypoint has the same arrival radius of 12 u (`TARGET_RADIUS`), and the craft is at a waypoint when, at the end of a physics substep, both its distance to the waypoint's current position is under 12 u and its speed relative to the waypoint's current velocity is under the waypoint's tolerance, which is 3 u/s (`RENDEZVOUS_V_TOL`) for a rendezvous and 10^9 u/s, i.e. any speed, for a flythrough. Both conditions are evaluated against the waypoint's state at that substep, so a rendezvous with a station is a velocity match with the station's orbital motion and a rendezvous with a point hung off a giant is a match with the giant's.

A tour is flown in slot order unless the instance's `order_free` flag is set (with probability

Kind	Where it is	How it moves	Velocity condition
free-space point	a fixed position in the level frame, sampled as above (`wp_anchor` $= -1$)	it does not; its velocity is zero	flythrough only
station	the position of a station body (massless, zero radius, on its own orbit around its host)	with the station	flythrough or rendezvous with the station's velocity
attached to a body	the body's position plus a constant offset vector of length (radius + atmosphere + 35 u) at a random angle, or, for a contested waypoint, of length 0.8–1.3× the body's equal-pull radius (Section 3.2)	the offset translates with the body and does not rotate with it	flythrough or rendezvous with the body's velocity

Table 10: The three waypoint kinds. Every kind has the same 12 u arrival radius; black holes and tracers are never anchors, and a body that already carries a waypoint is not chosen again in the same tour. The figure of the task (Figure 2d) shows the same conditions as four cells by separating the velocity condition from the attachment.

`p_order_free`, zero in training until the curriculum raises it and 0.3 on the blind suite), and the difference between the two is only in which waypoint an arrival is credited to: in a fixed-order tour, reaching a waypoint counts only if it is the next unvisited one in slot order, so that passing through the third waypoint on the way to the first does nothing, while in an order-free tour any unvisited waypoint counts whenever its conditions are met. The observation's notion of the current target (Appendix A) follows the same rule, being the next unvisited waypoint in a fixed-order tour and the nearest unvisited one in an order-free tour, and so does the potential used for reward shaping, which is the negative of the distance to the current target plus a flat allowance for each further waypoint still outstanding. The scoring of a completed tour is identical in the two cases (the same per-waypoint and completion rewards, the same weighted cost over time, path, fuel and exposure), so an order-free tour is neither easier nor harder by construction; it is different in that the navigator, and not the instance, chooses the sequence.

The craft's loadout is drawn against the tour it has to fly. With d_k the straight-line length of the k-th leg (from the start position to the first waypoint and then from each waypoint to the next, in slot order and ignoring gravity, the craft's initial velocity and any motion of the waypoints), the Δv the tour is estimated to require is

$$\Delta v_{\text{req}} \;=\; \sum_{k} \begin{cases} \sqrt{2\, a_{\text{eng}}\, d_k} & \text{flythrough, i.e. accelerate across the whole leg,} \\ 2\sqrt{a_{\text{eng}}\, d_k} & \text{rendezvous, i.e. accelerate for half the leg and brake for half,} \end{cases} \tag{4}$$

which `deltav_estimate()` computes and which is also what the observation reports as the Δv required over the legs still pending. The engine acceleration a_{eng} is drawn log-uniformly from $[0.6, 3]$ u/s^2 (`SHIP_ACCEL_RANGE`, overridden per suite, to $[1.5, 3]$ on the easy suite and $[0.8, 2]$ on the hard suite, and started generous by the training curriculum), the budget multiple β is drawn uniformly from $[0.55, 1.35]$ (`DELTAV_BUDGET_RANGE`, overridden per suite as Table 3 lists), and the tank, in seconds of full-throttle burn, is $\beta\, \Delta v_{\text{req}}/a_{\text{eng}}$ clipped to $[3, 60]$ s (`SHIP_FUEL_LIMITS`), so that the tank actually carried can be a larger multiple of the estimate than β on a very short tour and a smaller one on a very long tour. The reference scales that normalize the four cost terms are set from the same quantities, with the time and exposure references equal to $\sum_k 2\sqrt{d_k/a_{\text{eng}}}$, the path reference equal to $\sum_k d_k$ and the fuel reference equal to half the tank, and the objective

weights are either a single objective (with probability 0.5, drawn with probabilities 0.4, 0.2, 0.3 and 0.1 over time, path, fuel and exposure) or a Dirichlet mixture of them, with exposure available only on a level that has at least one zone, the zone charged for being drawn from the level's active zones with sensor zones favored by a factor of e over the rest.

An episode ends in exactly one of five ways, tested in this order at the end of every physics substep (60 Hz) and reported as the status code the evaluation suites count: `crashed`, the craft's position is inside the radius of a solid body (a station or a tracer, having no radius, cannot be crashed into); `destroyed`, health has reached zero under the accumulated damage from stellar radiation, atmospheric heating, debris and radiation belts; `arrived`, every active waypoint has been visited, which is the only success; `lost`, the craft is farther than 1.35 times the level radius from the origin; and, tested once per control tick rather than per substep, `timeout`, the tick count has reached $T_{\max}/\Delta t$ (2250 at the 150 s clock) with the tour unfinished. An empty tank is not among them. The burn in a substep is the smaller of the commanded burn and the fuel left, so that once the tank is empty the throttle does nothing, the craft coasts under gravity alone, and the episode continues until one of the five endings occurs, which on an energy-efficient path is often the clock (Section 3.6) and on a decaying one is a crash. The hazard terms themselves do not end an episode; they lower health, and only health reaching zero does.

D Reproduction

Everything reported in this paper is produced by scripts in the repository, from a checkout with the `spacenav` package installed and the GPU build of JAX, in the order below; `data/` is not under version control, and the level pools are regenerated from their seed blocks (seeds from 0 for the training pool, from 10^7 for the seen-family validation pool, from 2×10^7 for the holdout pool, from 3×10^7, 4×10^7 and 5×10^7 for the three long-horizon pools: `long`, the `long_val` pool the 600 s suites are drawn from, and the 900-level pool validated to 9,000 s) so that the levels themselves are reproducible without being distributed. One caveat: the standard pools were built when `LEVEL_HORIZON` in `spacenav/constants.py` was 300 s, and the constant has since been raised to 900 s for the long-horizon pools, so regenerating the standard pools bit for bit requires setting it back to 300 s first; at 900 s the same seeds yield a different accepted set with 38 snapshots per level.

```
# level pools: train, val_seen, val_holdout (data/pools/*.npz), then the
# long-horizon pool used at a 600 s clock
python scripts/build_pools.py --train 2000 --val 300 --holdout 400
SPACENAV_EPISODE_S=600 python scripts/build_pools.py --only long --n 200

# one training run at the 150 s clock; the frozen suites
# (data/eval/suites.pkl) are built on first use
SPACENAV_INSTRUMENTS=eta,stretch \
  python scripts/train.py --run abl_info_s0 --updates 3000 --seed 0 --pool train
SPACENAV_INSTRUMENTS=0 \
  python scripts/train.py --run ppo_v5_noinstr --updates 3000 --seed 0
SPACENAV_INSTRUMENTS=1 \
  python scripts/train.py --run ppo_v4 --updates 3000 --seed 0
SPACENAV_INSTRUMENTS=budget,demand \
  python scripts/train.py --run abl_redundant_s0 --updates 3000 --seed 0

# the 600 s clock: suites from the long_val pool (built here on first use),
```

```
# then training on the long pool against them
SPACENAV_EPISODE_S=600 python scripts/build_long_suites.py --clock 600
SPACENAV_EPISODE_S=600 SPACENAV_SUITES=data/eval/suites_long.pkl \
  python scripts/train.py --run long600_s0 --updates 3000 --seed 0 --pool long

# the clock sweep (the reference optimizer at 150, 300, 600 and 900 s)
bash scripts/clock_sweep.sh data/curriculum/clock

# the ghost fleets: every checkpoint of one run on one instance, rolled out
# with the instrument setting the run was trained with
SPACENAV_INSTRUMENTS=eta,stretch python scripts/ghost_fleet.py \
  --run abl_info_s0 --suite medium --task 1 --rollouts 16 \
  --out data/ghosts/medium_1.json

# the zero-thrust navigator to 900 s, the 9,000 s pool, and the same to 9,000 s
python scripts/noop_clock.py measure --clock 900
python scripts/noop_clock.py build10x --per-family 100
python scripts/noop_clock.py measure10x --clock 9000
python scripts/noop_clock.py figure

# per-instance success against eta, A, D and family
python scripts/correlations.py

# the training-budget run
SPACENAV_INSTRUMENTS=eta,stretch \
  python scripts/train.py --run budget10k_s0 --updates 10000 --seed 0

# figures
python scripts/paper_figures.py --which ghosts,ablation,clock,curves,eta
python scripts/fig_task.py --suite hard --task 63 --run ppo_v5_noinstr
python scripts/fig_phase.py compute --pool data/pools/long.npz
python scripts/fig_phase.py render
```

Two environment variables select the process-wide settings that the body of the paper varies. `SPACENAV_EPISODE_S` sets the episode clock in seconds (default `150.0`) and is read once, at import, by `spacenav/constants.py`, from which the tick count, the time-remaining entries of the observation and the number of snapshots a pool stores are all derived; a clock longer than the window a pool was validated over (300 s for `train` and the two validation pools, 900 s for the long-horizon pools) runs late-starting episodes past where the level was checked, which is why the long clock has its own pools and its own suites, selected with `SPACENAV_SUITES`, and why `train.py` refuses to build a non-default suite file on its own. `SPACENAV_INSTRUMENTS` selects which of the four instrument channels are live (Appendix A), with the training default `eta,stretch`, and since every setting produces the same observation shape a checkpoint must be evaluated or replayed under the setting recorded in its run's `config.json`, as nothing will complain if it is not. Every evaluation reported here is the greedy policy (the most probable action at each tick) on the frozen suites, written by `train.py` to `eval.jsonl` at every fiftieth update, with the quoted success rate being the mean of a run's last three checkpoints; the ghost-fleet replays are the one place actions are sampled.

The simulation, environment, optimizer and training loop are written in Python 3.12 against JAX 0.9.2 (the `jax[cuda12]` build), with the policy network in Flax 0.12.6 and the optimizer state in Optax 0.2.8; array handling and the statistics in the figure scripts use NumPy 2.4.3 and SciPy

1.17.1, and the figures are drawn with matplotlib 3.11.2, all as pinned in `requirements-gpu.txt` and installed in the conda environment the runs were made in. The browser client in `web/` is a direct port of the physics in JavaScript, and a parity test (`tests/test_js_parity.py`) holds the two integrators to the same trajectories, which is what makes the replays on the companion site the flights that were scored rather than re-renderings of them.

The twenty headline runs were made on two cards. Seeds 0 and 1 of the no-channel and all-four arms ran on an NVIDIA GeForce GTX 1080 Ti (11 GB, Pascal), at about 17,000 environment steps per second averaged over a whole run including its checkpoint evaluations (the training loop alone measures 31,900 steps/s at this network width), or 6.4 h for 3,000 updates; the remaining sixteen runs, and all of the long-clock runs, ran on an NVIDIA GeForce RTX 3090 (24 GB) at about 111,000 steps per second for a single process, or 67 min per run, a card that one such process already saturates, so that two concurrent runs together deliver three quarters of the throughput of one. Within the no-channel arm, whose seeds are split across the two cards, no difference attributable to the card was found. The cross-entropy optimizer, the chaos probe and the phase portraits run on either card, with the single-burn outcome maps of Figure 4 being the one computation (368,640 full-episode rollouts per level) that is impractical on the smaller one.

The reasoning behind every design choice, rejected approach and measurement that changed the plan is recorded in `docs/decisions/`, as numbered, append-only decision records; a reversed decision is superseded by a new record rather than edited, so that the wrong turns remain legible, and each record cites the script, sample size or paper that forced it. The records are the trail from which the arguments of Sections 3.2 through 5 were assembled (for instance, the equal-pull surface replacing the resonance-overlap scale, the solvability yardstick being judged on sample size, the instrument channels being split along the redundancy line, and the clock becoming a per-process setting), and they are the place to look when a number in this paper and a number in the repository disagree, since the record will say which measurement superseded which.

References

[1] Henri Poincaré. Sur le problème des trois corps et les équations de la dynamique. *Acta Mathematica*, 13:1–270, 1890. Crossref indexes the memoir chapter by chapter; the introduction is doi:10.1007/BF02392506.

[2] Victor G. Szebehely. *Theory of Orbits: The Restricted Problem of Three Bodies*. Academic Press, New York, 1967. ISBN 978-0-12-680650-2.

[3] Michel Hénon and Carl Heiles. The applicability of the third integral of motion: Some numerical experiments. *The Astronomical Journal*, 69:73–79, 1964. doi: 10.1086/109234.

[4] Roger R. Bate, Donald D. Mueller, and Jerry E. White. *Fundamentals of Astrodynamics*. Dover, New York, 1971. ISBN 978-0-486-60061-1.

[5] Richard H. Battin. *An Introduction to the Mathematics and Methods of Astrodynamics*. American Institute of Aeronautics and Astronautics, Reston, VA, revised edition, 1999. ISBN 978-1-56347-342-5.

[6] George William Hill. Researches in the lunar theory. *American Journal of Mathematics*, 1 (1):5–26, 1878. doi: 10.2307/2369430.

[7] Gary A. Flandro. Fast reconnaissance missions to the outer solar system utilizing energy derived from the gravitational field of Jupiter. *Astronautica Acta*, 12(4):329–337, 1966.

[8] C. E. Kohlhase and P. A. Penzo. Voyager mission description. *Space Science Reviews*, 21 (2):77–101, 1977. doi: 10.1007/BF00200846.

[9] NASA Science. Voyager: Planetary voyage. https://science.nasa.gov/mission/voyager/planetary-voyage/, . "the flight time to Neptune was reduced from 30 years to 12"; accessed 2026-11-01.

[10] N. J. Fox, M. C. Velli, S. D. Bale, R. Decker, A. Driesman, R. A. Howard, J. C. Kasper, J. Kinnison, M. Kusterer, D. Lario, M. K. Lockwood, D. J. McComas, N. E. Raouafi, and A. Szabo. The Solar Probe Plus mission: Humanity's first visit to our star. *Space Science Reviews*, 204(1–4):7–48, 2016. doi: 10.1007/s11214-015-0211-6.

[11] Yanping Guo. Solar Probe Plus: Mission design challenges and trades. *Acta Astronautica*, 67(9–10):1063–1072, 2010. doi: 10.1016/j.actaastro.2010.06.007.

[12] Amelia L. Batcha, Jacob Williams, Timothy F. Dawn, Jeffrey P. Gutkowski, Maxon V. Widner, Sarah L. Smallwood, Brian J. Killeen, Elizabeth C. Williams, and Robert E. Harpold. Artemis I trajectory design and optimization. In *AAS/AIAA Astrodynamics Specialist Conference*, 2020. Paper AAS 20-649. NASA NTRS 20205005150, https://ntrs.nasa.gov/citations/20205005150.

[13] NASA Science. Voyager 2. https://science.nasa.gov/mission/voyager/voyager-2/, . Launch and closest-approach dates; accessed 2026-11-01.

[14] NASA Science. Parker solar probe. https://science.nasa.gov/mission/parker-solar-probe/, . Launch 2018-08-12; accessed 2026-11-01.

[15] NASA Science. Final Venus flyby for NASA's Parker Solar Probe queues closest Sun pass. https://science.nasa.gov/science-research/heliophysics/final-venus-flyby-for-nasas-parker-solar-probe-queues-closest-sun-pass, 2024. Flyby 2024-11-06; perihelion 2024-12-24.

[16] NASA. Orion will go the distance in retrograde orbit during Artemis I. https://www.nasa.gov/missions/orion-will-go-the-distance-in-retrograde-orbit-during-artemis-i/. accessed 2026-11-01.

[17] NASA. Artemis I – flight day 20: Orion conducts return powered flyby. https://www.nasa.gov/blogs/missions/2022/12/05/artemis-i-flight-day-20-orion-conducts-return-powered-flyby/, 2022.

[18] Dario Izzo, Marcus Märtens, and Binfeng Pan. A survey on artificial intelligence trends in spacecraft guidance dynamics and control. *Astrodynamics*, 3(4):287–299, 2019. doi: 10.1007/s42064-018-0053-6.

[19] Dario Izzo and Ekin Öztürk. Real-time guidance for low-thrust transfers using deep neural networks. *Journal of Guidance, Control, and Dynamics*, 44(2):315–327, 2021. doi: 10.2514/1.G005254.

[20] Brian Gaudet, Richard Linares, and Roberto Furfaro. Deep reinforcement learning for six degree-of-freedom planetary landing. *Advances in Space Research*, 65(7):1723–1741, 2020. doi: 10.1016/j.asr.2019.12.030.

[21] Nicholas B. LaFarge, Daniel Miller, Kathleen C. Howell, and Richard Linares. Autonomous closed-loop guidance using reinforcement learning in a low-thrust, multi-body dynamical environment. *Acta Astronautica*, 186:1–23, 2021. doi: 10.1016/j.actaastro.2021.05.014.

[22] Stefano Bonasera, Natasha Bosanac, Christopher J. Sullivan, Ian Elliott, Nisar Ahmed, and Jay W. McMahon. Designing Sun–Earth L2 halo orbit stationkeeping maneuvers via reinforcement learning. *Journal of Guidance, Control, and Dynamics*, 46(2):301–311, 2023. doi: 10.2514/1.G006783.

[23] Karl Cobbe, Oleg Klimov, Chris Hesse, Taehoon Kim, and John Schulman. Quantifying generalization in reinforcement learning. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

[24] Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76: 201–264, 2023. doi: 10.1613/jair.1.14174.

[25] Edward N. Lorenz. Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20(2):130–141, 1963. doi: 10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2.

[26] Celso Grebogi, Steven W. McDonald, Edward Ott, and James A. Yorke. Final state sensitivity: An obstruction to predictability. *Physics Letters A*, 99(9):415–418, 1983. doi: 10.1016/0375-9601(83)90945-3.

[27] Ian M. Mitchell, Alexandre M. Bayen, and Claire J. Tomlin. A time-dependent Hamilton–Jacobi formulation of reachable sets for continuous dynamic games. *IEEE Transactions on Automatic Control*, 50(7):947–957, 2005. doi: 10.1109/TAC.2005.851439.

[28] D. Q. Mayne, M. M. Seron, and S. V. Raković. Robust model predictive control of constrained linear systems with bounded disturbances. *Automatica*, 41(2):219–224, 2005. doi: 10.1016/j.automatica.2004.08.019.

[29] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X.

[30] George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung. *Time Series Analysis: Forecasting and Control*. Wiley, Hoboken, NJ, 5 edition, 2015.

[31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017. URL `https://arxiv.org/abs/1706.03762`.

[32] Edward A. Belbruno and James K. Miller. Sun-perturbed Earth-to-Moon transfers with ballistic capture. *Journal of Guidance, Control, and Dynamics*, 16(4):770–775, 1993. doi: 10.2514/3.21079.

[33] Wang Sang Koon, Martin W. Lo, Jerrold E. Marsden, and Shane D. Ross. Heteroclinic connections between periodic orbits and resonance transitions in celestial mechanics. *Chaos*, 10(2):427–469, 2000. doi: 10.1063/1.166509.

[34] Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. Model-based reinforcement learning: A survey. *Foundations and Trends in Machine Learning*, 16(1): 1–118, 2023. URL `https://arxiv.org/abs/2006.16712`.

[35] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018. ISBN 978-0-262-03924-6.

[36] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint, 2017. Accessed 2026-10-30.

[37] Tao Wang, Sylvia Herbert, and Sicun Gao. Fractal landscapes in policy optimization. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[38] Luke Metz, C. Daniel Freeman, Samuel S. Schoenholz, and Tal Kachman. Gradients are not all you need. arXiv preprint, 2021. Accessed 2026-10-30.

[39] Paavo Parmas, Carl Edward Rasmussen, Jan Peters, and Kenji Doya. PIPPS: Flexible model-based policy search robust to the curse of chaos. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[40] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 41–48, 2009. doi: 10.1145/1553374.1553380.

[41] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181):1–50, 2020.

[42] Hiroki Furuta, Tatsuya Matsushima, Tadashi Kozuno, Yutaka Matsuo, Sergey Levine, Ofir Nachum, and Shixiang Shane Gu. Policy information capacity: Information-theoretic measure for task complexity in deep reinforcement learning. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

[43] Michael Dennis, Natasha Jaques, Eugene Vinitsky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[44] Alexander Rutherford, Michael Beukman, Timon Willi, Bruno Lacerda, Nick Hawes, and Jakob Foerster. No regrets: Investigating and improving regret approximations for curriculum discovery. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.

[45] Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, Cambridge, 2006. ISBN 978-0-521-86205-9. Free online at `http://lavalle.pl/planning/`.

[46] John H. Reif. Complexity of the mover's problem and generalizations. In *20th Annual Symposium on Foundations of Computer Science (SFCS 1979)*, pages 421–427, 1979. doi: 10.1109/SFCS.1979.10.

[47] John E. Hopcroft, Jacob T. Schwartz, and Micha Sharir. On the complexity of motion planning for multiple independent objects; PSPACE-hardness of the "warehouseman's problem". *The International Journal of Robotics Research*, 3(4):76–88, 1984. doi: 10.1177/027836498400300405.

[48] John F. Canny. *The Complexity of Robot Motion Planning*. MIT Press, Cambridge, MA, 1988. ISBN 978-0-262-03136-3.

[49] John Reif and Micha Sharir. Motion planning in the presence of moving obstacles. *Journal of the ACM*, 41(4):764–790, 1994. doi: 10.1145/179812.179911.

[50] Christos H. Papadimitriou and John N. Tsitsiklis. The complexity of Markov decision processes. *Mathematics of Operations Research*, 12(3):441–450, 1987. doi: 10.1287/moor.12.3.441.

[51] Omid Madani, Steve Hanks, and Anne Condon. On the undecidability of probabilistic planning and related stochastic optimization problems. *Artificial Intelligence*, 147(1–2):5–34, 2003. doi: 10.1016/S0004-3702(02)00378-8.

[52] Cristopher Moore. Unpredictability and undecidability in dynamical systems. *Physical Review Letters*, 64(20):2354–2357, 1990. doi: 10.1103/PhysRevLett.64.2354.

[53] Thomas A. Henzinger, Peter W. Kopke, Anuj Puri, and Pravin Varaiya. What's decidable about hybrid automata? *Journal of Computer and System Sciences*, 57(1):94–124, 1998. doi: 10.1006/jcss.1998.1581.

[54] Vincent D. Blondel and John N. Tsitsiklis. A survey of computational complexity results in systems and control. *Automatica*, 36(9):1249–1274, 2000. doi: 10.1016/S0005-1098(00)00050-9.

[55] Bruce Donald, Patrick Xavier, John Canny, and John Reif. Kinodynamic motion planning. *Journal of the ACM*, 40(5):1048–1066, 1993. doi: 10.1145/174147.174150.

[56] Stephen M. Hammel, James A. Yorke, and Celso Grebogi. Do numerical orbits of chaotic dynamical processes represent true orbits? *Journal of Complexity*, 3(2):136–145, 1987. doi: 10.1016/0885-064X(87)90024-0.

[57] Tim Sauer, Celso Grebogi, and James A. Yorke. How long do numerical chaotic solutions remain valid? *Physical Review Letters*, 79(1):59–62, 1997. doi: 10.1103/PhysRevLett.79.59.

[58] Timothy D. Sauer. Shadowing breakdown and large errors in dynamical simulations of physical systems. *Physical Review E*, 65(3):036220, 2002. doi: 10.1103/PhysRevE.65.036220.

[59] Tjarda Boekholt and Simon Portegies Zwart. On the reliability of N-body simulations. *Computational Astrophysics and Cosmology*, 2:2, 2015. doi: 10.1186/s40668-014-0005-3.

[60] Simon F. Portegies Zwart and Tjarda C. N. Boekholt. Numerical verification of the microscopic time reversibility of Newton's equations of motion: Fighting exponential divergence. *Communications in Nonlinear Science and Numerical Simulation*, 61:160–166, 2018. doi: 10.1016/j.cnsns.2018.02.002.

[61] Jack Wisdom and Matthew Holman. Symplectic maps for the N-body problem. *The Astronomical Journal*, 102:1528–1538, 1991. doi: 10.1086/115978.

[62] Hanno Rein and David S. Spiegel. IAS15: a fast, adaptive, high-order integrator for gravitational dynamics, accurate to machine precision over a billion orbits. *Monthly Notices of the Royal Astronomical Society*, 446(2):1424–1437, 2015. doi: 10.1093/mnras/stu2164.

[63] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[64] Niels Justesen, Ruben Rodriguez Torrado, Philip Bontrager, Ahmed Khalifa, Julian Togelius, and Sebastian Risi. Illuminating generalization in deep reinforcement learning through procedural level generation. arXiv preprint, 2018. Accessed 2026-10-30.

[65] Boris V. Chirikov. A universal instability of many-dimensional oscillator systems. *Physics Reports*, 52(5):263–379, 1979. doi: 10.1016/0370-1573(79)90023-1.

[66] Jack Wisdom. The resonance overlap criterion and the onset of stochastic behavior in the restricted three-body problem. *The Astronomical Journal*, 85:1122–1133, 1980. doi: 10.1086/112778.

[67] Katherine M. Deck, Matthew Payne, and Matthew J. Holman. First-order resonance overlap and the stability of close two-planet systems. *The Astrophysical Journal*, 774(2):129, 2013. doi: 10.1088/0004-637X/774/2/129.

[68] Bruno H. Repp. Sensorimotor synchronization: A review of the tapping literature. *Psychonomic Bulletin & Review*, 12(6):969–992, 2005. doi: 10.3758/BF03206433.

[69] David L. Woods, John M. Wyma, E. William Yund, Timothy J. Herron, and Bruce Reed. Factors influencing the latency of simple reaction time. *Frontiers in Human Neuroscience*, 9:131, 2015. doi: 10.3389/fnhum.2015.00131.

[70] David W. Franklin and Daniel M. Wolpert. Computational mechanisms of sensorimotor control. *Neuron*, 72(3):425–442, 2011. doi: 10.1016/j.neuron.2011.10.006.

[71] David W. Franklin, Sae Franklin, and Daniel M. Wolpert. Fractionation of the visuomotor feedback response to directions of movement and perturbation. *Journal of Neurophysiology*, 112(9):2218–2233, 2014. doi: 10.1152/jn.00377.2013.

[72] Giancarlo Benettin, Luigi Galgani, Antonio Giorgilli, and Jean-Marie Strelcyn. Lyapunov characteristic exponents for smooth dynamical systems and for Hamiltonian systems; a method for computing all of them. Part 1: Theory. *Meccanica*, 15(1):9–20, 1980. doi: 10.1007/BF02128236.

[73] Edward Ott. *Chaos in Dynamical Systems*. Cambridge University Press, Cambridge, 2nd edition, 2002. ISBN 978-0-521-01084-9.

[74] M. A. Bucci, O. Semeraro, A. Allauzen, G. Wisniewski, L. Cordier, and L. Mathelin. Control of chaotic systems by deep reinforcement learning. *Proceedings of the Royal Society A*, 475 (2231):20190351, 2019. doi: 10.1098/rspa.2019.0351.

[75] Jean Rabault, Miroslav Kuchta, Atle Jensen, Ulysse Réglade, and Nicolas Cerardi. Artificial neural networks trained through deep reinforcement learning discover control strategies for active flow control. *Journal of Fluid Mechanics*, 865:281–302, 2019. doi: 10.1017/jfm.2019.62.

[76] C. Vignon, J. Rabault, and R. Vinuesa. Recent advances in applying deep reinforcement learning for flow control: Perspectives and future directions. *Physics of Fluids*, 35(3):031301, 2023. doi: 10.1063/5.0143913.

[77] Edward Ott, Celso Grebogi, and James A. Yorke. Controlling chaos. *Physical Review Letters*, 64(11):1196–1199, 1990. doi: 10.1103/PhysRevLett.64.1196.

[78] Troy Shinbrot, Edward Ott, Celso Grebogi, and James A. Yorke. Using chaos to direct trajectories to targets. *Physical Review Letters*, 65(26):3215–3218, 1990. doi: 10.1103/PhysRevLett.65.3215.

[79] Steven W. McDonald, Celso Grebogi, Edward Ott, and James A. Yorke. Fractal basin boundaries. *Physica D: Nonlinear Phenomena*, 17(2):125–153, 1985. doi: 10.1016/0167-2789(85)90001-6.

[80] Jacobo Aguirre, Ricardo L. Viana, and Miguel A. F. Sanjuán. Fractal structures in nonlinear dynamics. *Reviews of Modern Physics*, 81(1):333–386, 2009. doi: 10.1103/RevModPhys.81.333.

[81] Andrey N. Kolmogorov. On conservation of conditionally periodic motions for a small change in Hamilton's function. *Doklady Akademii Nauk SSSR*, 98:527–530, 1954. English translation in *Lecture Notes in Physics* 93, 51–56 (1979), doi:10.1007/BFb0021737.

[82] Vladimir I. Arnol'd. Proof of a theorem of A. N. Kolmogorov on the invariance of quasi-periodic motions under small perturbations of the Hamiltonian. *Russian Mathematical Surveys*, 18(5):9–36, 1963. doi: 10.1070/RM1963v018n05ABEH004130.

[83] N. N. Nekhoroshev. An exponential estimate of the time of stability of nearly-integrable Hamiltonian systems. *Russian Mathematical Surveys*, 32(6):1–65, 1977. doi: 10.1070/RM1977v032n06ABEH003859.

[84] Jacques Laskar. A numerical experiment on the chaotic behaviour of the Solar System. *Nature*, 338(6212):237–238, 1989. doi: 10.1038/338237a0.

[85] Gerald Jay Sussman and Jack Wisdom. Chaotic evolution of the Solar System. *Science*, 257(5066):56–62, 1992. doi: 10.1126/science.257.5066.56.

[86] Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999. doi: 10.1023/A:1010091220143.

[87] Grady Williams, Andrew Aldrich, and Evangelos A. Theodorou. Model predictive path integral control: From theory to parallel computation. *Journal of Guidance, Control, and Dynamics*, 40(2):344–357, 2017. doi: 10.2514/1.G001921.

[88] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021.

[89] Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How many random seeds? Statistical power analysis in deep reinforcement learning experiments. arXiv preprint, 2018. Accessed 2026-10-30.

[90] Hojoon Lee, Youngdo Lee, Takuma Seno, Donghu Kim, Peter Stone, and Jaegul Choo. Hyperspherical normalization for scalable deep reinforcement learning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. The method is named SimbaV2.

[91] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[92] Michael Matthews, Michael Beukman, Chris Lu, and Jakob Foerster. Kinetix: Investigating the training of general agents through open-ended physics-based control tasks. In *International Conference on Learning Representations (ICLR)*, 2025.

[93] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2018.

[94] Lucy Lai and Samuel J. Gershman. Human decision making balances reward maximization and policy compression. *PLOS Computational Biology*, 20(4):e1012057, 2024. doi: 10.1371/journal.pcbi.1012057.

[95] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[96] Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL^2: Fast reinforcement learning via slow reinforcement learning. arXiv preprint, 2016. Accessed 2026-10-30.

[97] Jacob Beck, Risto Vuorio, Evan Zheran Liu, Zheng Xiong, Luisa Zintgraf, Chelsea Finn, and Shimon Whiteson. A tutorial on meta-reinforcement learning. *Foundations and Trends in Machine Learning*, 18(2–3):224–384, 2025. doi: 10.1561/2200000080.

[98] Rémy Portelas, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. Automatic curriculum learning for deep RL: A short survey. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 4819–4825, 2020. doi: 10.24963/ijcai.2020/671.

[99] Carlos Florensa, David Held, Xinyang Geng, and Pieter Abbeel. Automatic goal generation for reinforcement learning agents. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[100] Tambet Matiisen, Avital Oliver, Taco Cohen, and John Schulman. Teacher–student curriculum learning. *IEEE Transactions on Neural Networks and Learning Systems*, 31(9): 3732–3740, 2020. doi: 10.1109/TNNLS.2019.2934906.

[101] Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

[102] Minqi Jiang, Michael Dennis, Jack Parker-Holder, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Replay-guided adversarial environment design. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021.

[103] Jack Parker-Holder, Minqi Jiang, Michael Dennis, Mikayel Samvelyan, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Evolving curricula with regret-based environment design. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.

[104] Minqi Jiang, Michael Dennis, Jack Parker-Holder, Andrei Lupu, Heinrich Küttler, Edward Grefenstette, Tim Rocktäschel, and Jakob Foerster. Grounding aleatoric uncertainty for unsupervised environment design. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.

[105] H. J. Terry Suh, Max Simchowitz, Kaiqing Zhang, and Russ Tedrake. Do differentiable simulators give better policy gradients? In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.

[106] Torsten Heinze. Master the complexity of spaceflight. YouTube video, channel *braintruffle*, `https://www.youtube.com/watch?v=dhYqflvJMXc`, 2024. Published 2024-01-25, 32 min. Accessed 2026-10-30.

[107] Torsten Heinze. Simulating Jupiter's comet shield. YouTube video, channel *braintruffle*, `https://www.youtube.com/watch?v=1zu41rrc_Ng`, 2024. Published 2024-11-07, 19 min. Accessed 2026-10-30.

[108] Torsten Heinze. The code that revolutionized orbital simulation. YouTube video, channel *braintruffle*, `https://www.youtube.com/watch?v=nCg3aXn5F3M`, 2025. Published 2025-12-11, 28 min. Accessed 2026-10-30.

[109] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112 (1–2):181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1.